

NATIONAL ECONOMICS UNIVERSITY

Faculty of Mathematical Economics

-----***-----



BACHELOR THESIS

Transfer learning for Credit scoring:

***An experimental study of Heterogeneous feature
augmentation and Domain adversarial learning***

Name: Phung Bao Ngoc

Class: Actuary Science and Risk Management 64

Student's ID: 11224762

Supervisor: Dr. Nguyen Manh The

Lecturer Nguyen Quoc Thanh

Hanoi, May 2026

ACKNOWLEDGEMENTS

I would like to express my gratitude to my supervisors, Dr. Nguyen Manh The & Visiting lecturer Nguyen Quoc Thanh. Their guidance, thoughtful feedback, and constant encouragement have been invaluable throughout the course of this thesis. Their insights on heterogeneous domain adaptation, the design of negative-transfer ablation experiments, and the interpretation of GRL failure modes greatly strengthened both the research framework and the analysis of results.

I am deeply thankful to the faculty members of the Faculty of Mathematical Economics. Over the past four years, their teaching has provided me with the theoretical, statistical, and methodological foundation that made this work possible.

My appreciation goes to my family and friends, whose unwavering support, patience, and encouragement have sustained me during this journey. Their belief in me has been a source of strength and motivation.

Finally, I would like to extend my sincere gratitude to the colleagues at the Risk – Anti-Money Laundering & FATCA Department of VIB. Their support and practical insights into risk management and anti-money laundering practices have been invaluable in shaping the applied perspective of this research on banking risk.

Without the guidance, encouragement, and support of my supervisor, faculty members, family, friends, and colleagues, this thesis would not have been possible.

SUMMARY

This thesis presents an experimental study of two transfer learning techniques (both belong to heterogeneous domain adaptation methods). These two methods - Heterogeneous feature augmentation (HFA) and Domain-adversarial neural network (DANN) were applied to solve credit scoring problem. The source domain is the Lending Club dataset (2,26 million US loan records, 2007–2018) and the target domain is the Taiwan Default of Credit Card Clients dataset (30000 Taiwanese cardholders, 2005), with both domains differing substantially in feature dimensionality and marginal distributions.

HFA is formalized following Yang et al. (2020, Eq. 6.8) and solved via Block Coordinate Descent with closed-form Frobenius. DANN is implemented as a HeteroDANN with separate domain encoders connected through a Gradient Reversal Layer, following Ganin et al. (2016). Both methods are evaluated against three baselines (Logistic Regression, Random Forest, XGBoost) under 5-fold cross-validation, independent hold-out testing, and a structured 3-case ablation designed to detect Negative Transfer explicitly.

Experimental results show that neither transfer method surpasses the strongest baseline. Findings reveal that HFA achieves stable but limited gains, while DANN suffers from GRL collapse under full feature sets. These outcomes highlight the limitations of adversarial alignment under label shift and the relative robustness of linear alignment under severe domain gaps.

The thesis provides a benchmark, evidence of failure modes under large domain gaps, and a recommendation for explicit negative-transfer ablation in credit risk research.

Table of Contents

LIST OF TABLES	7
Chapter 1. Introduction	8
1.1. Research Background	8
1.2. Research Problem	9
1.2.1. Heterogeneous Domain Adaptation	9
1.2.2. What to transfer	10
1.2.3. Where to transfer	10
1.2.4. When to transfer	11
1.2.5. How to transfer	11
1.2.6. Negative Transfer Problem	11
1.3. Research Questions	12
Chapter 2. Literature Review	13
2.1. Overview of Transfer Learning	13
2.1.1. Definition and Taxonomy	13
2.1.2. Domain Adaptation and H-divergence Theory	13
2.1.3. Heterogeneous Domain Adaptation	14
2.1.4. Adversarial Domain Adaptation	14
2.2. Overview of Credit Scoring	15
2.2.1. Classical Statistical Era (1940–1990)	15
2.2.2. Classical Machine Learning Era (1990–2010)	15
2.2.3. Modern Machine Learning and Deep Learning (2010 to present)	15
2.3. Overview of applying Transfer Learning for Credit Scoring	15
2.3.1. Homogeneous Transfer Learning	15
2.3.2. Heterogeneous Transfer Learning	16
2.3.3. Adversarial and GAN-based Methods in Credit Scoring	16
2.4. Negative Transfer	16
2.5. Research Gap	16
Chapter 3. Datasets and Domain Analysis	18
3.1. Target Domain - Taiwan Default of Credit Card Clients (TDCCC)	18
3.1.1. Feature description	18
3.1.2. Taiwan Default of Credit Card Clients preprocessing	18
3.2. Source Domain - Lending Club (LC)	19

3.2.1. Feature description.....	19
3.2.2. Lending Club preprocessing	21
3.2.3. Lending Club Feature selection cases.....	22
3.2.4. Stratified subsampling of Lending Club	23
3.3. Domain Comparison	23
Chapter 4. Research Methodology.....	25
4.1. Baseline models	25
4.1.1. Models' selection	25
4.1.2. Data splitting.....	25
4.1.3. Baseline experimental design	25
4.1.4. Model evaluation	26
4.2. Heterogeneous Feature Augmentation (HFA) Methodology	28
4.2.1. HFA theoretical foundation	28
4.2.2. Objective function – Equation 6.8 (Yang, 2020).....	29
4.2.3. Frobenius projection	29
4.2.4. Johnson–Lindenstrauss (JL) random projection initialization.....	29
4.2.5. Subgradient computation (P & Q)	30
4.2.6. Block Coordinate Descent	31
4.2.7. Platt probability calibration.....	31
4.2.8. HFA experimental design	31
4.3. Domain-Adversarial Neural Network (DANN) Methodology	32
4.3.1. H-divergence theory.....	32
4.3.2. DANN Principle (Ganin et al., 2016)	33
4.3.3. Gradient Reversal Layer (GRL).....	34
4.3.4. HeteroDANN Architecture	34
4.3.5. Training Schedule (Ganin et al., 2016, §4.2).....	35
4.3.6. Combined Loss Function and Training Loop	36
4.3.7. Prediction and Evaluation	36
4.3.8. DANN experimental design.....	37
4.4. Negative Transfer Ablation Design	37
Chapter 5. Result, Discussion and Implications	39
5.1. Baseline results	39
5.1.1. Cross-validation results.....	39
5.1.2. Hold-out results.....	39

5.1.3. Baseline Analysis.....	40
5.2. HFA results	40
5.3. DANN Results	41
5.4. Ablation study results	43
5.4.1. HFA ablation results	43
5.4.2. DANN ablation results.....	44
5.4.3. Cross-method ablation comparison.....	45
5.5. Summary of key findings.....	45
5.5.1. Overall comparison.....	45
5.5.2. Root cause analysis of Negative transfer	47
Chapter 6. Conclusion.....	49
6.1. Research summary	49
6.2. HFA method evaluation	49
6.3. DANN method evaluation	50
6.4. HFA and DANN comparison.....	50
6.5. Limitations	51
6.6. Future research.....	51
REFERENCES	53
Appendix A - Mathematical Derivations	59
Appendix B - Experimental Design Rationale.....	64
Appendix C - Notation.....	66

LIST OF TABLES

Table 1: Target domain - Taiwan Default of Credit Card Clients feature description	18
Table 2: Source domain - Lending Club feature description	19
Table 3: Lending Club's selected feature set	22
Table 4: Domain comparison.....	23
Table 5: Baseline method - Logistic Regression's parameters	25
Table 6: Baseline method - Random Forest's parameters	26
Table 7: Baseline method - XGBoost's parameters	26
Table 8: HFA method's parameters	31
Table 9: Four modules of DANN	35
Table 10: DANN method's parameters	37
Table 11: Baseline method - Cross-validation (5-fold) results	39
Table 12: Baseline method - Hold-out results	39
Table 13: HFA method - Full LC results	40
Table 14: HFA method - Selected LC results	40
Table 15: DANN method - Full LC results	41
Table 16: DANN method - Selected LC results	42
Table 17: HFA ablation - Full LC results	43
Table 18: HFA ablation - Selected LC results	43
Table 19: DANN ablation - Full LC results.....	44
Table 20: DANN ablation - Selected LC results	44
Table 21: Ablation comparison.....	45
Table 22: Overall comparison (1)	45
Table 23: Overall comparison (2)	46
Table 24: Overall comparison (3)	46
Table 25: HFA and DANN result comparison	50

Chapter 1. Introduction

1.1. Research Background

Credit scoring is the process of turning a borrower's likelihood of default into a numerical score or probability, using information such as demographics, financial records, and repayment history. The methodological foundations of the field can be traced to the seminal work of Durand (1941), in which borrower characteristics were first used systematically to distinguish good from bad credit profiles in US consumer instalment lending. Later, Altman (1968) introduced the Z-score model through linear discriminant analysis, laying a quantitative foundation for predicting corporate bankruptcy. Hand and Henley (1997) later described credit scoring as "one of the most successful applications of statistics and machine learning in finance" in their comprehensive review in the "Journal of the Royal Statistical Society Series A".

After the 2008 global financial crisis, regulators tightened standards for internal credit risk systems. Under Basel III reforms (Basel Committee on Banking Supervision, 2017), banks adopting the Internal Ratings-Based (IRB) approach must estimate three key risk parameters: Probability of Default (PD), Loss Given Default (LGD), and Exposure at Default (EAD). This requirement pushed banks worldwide to invest heavily in credit modeling, while also raising expectations for accuracy and interpretability. At the same time, advances in machine learning have turned credit scoring into a dynamic interdisciplinary field spanning statistics, computer science, and finance.

In this context, Lessmann et al. (2015) conducted a large benchmark study comparing 41 classifiers across 8 standard credit datasets. Results showed that ensemble methods, especially Random Forest and Gradient Boosting, improved AUC by 1 to 3 percentage points over traditional logistic regression. However, they cautioned that these gains are reliable only when datasets are sufficiently large and well-labeled, conditions that are not always met in practice, particularly in emerging markets or during the early stages of a credit institution's operations.

In Vietnam, formal credit scoring developed alongside the restructuring of the banking sector as the country moved toward a market-oriented economy after 1987. The National Credit Information Center (CIC) was created in 1999 under the State Bank of Vietnam to collect, process, and share credit information for systemic risk management (State Bank of Vietnam, 1999). A major milestone came in 2015, when CIC introduced version 1.0 of its individual credit scoring model-the first in Vietnam to meet international standards. In April 2021, CIC released version 2.0, developed with support from the NICE Group (South Korea), to help credit institutions shift from collateral-based lending toward unsecured lending based on probabilistic risk forecasts (State Bank of Vietnam, 2021). Despite these advances, most domestic commercial banks continue to rely on their own internally developed models rather than directly adopting international ones (Nguyen et al., 2024).

On the regulatory side, the Law on Credit Institutions of 2010 (amended in 2017) sets out risk management principles and requires banks to establish internal credit rating systems (National Assembly of Vietnam, 2010). More recently, in 2024, the State Bank of Vietnam issued a draft decree introducing a regulatory sandbox for the banking sector, designed to provide a legal framework for fintech firms to test AI- and big-data-driven credit scoring solutions (State Bank of Vietnam, 2024).

Overall, this background reflects a transitional state of Vietnam's credit ecosystem: historical data remain fragmented, supervised labels are scarce, and customer segments show significant distributional gaps. These conditions make the application of Transfer Learning and Domain Adaptation techniques particularly relevant.

1.2. Research Problem

In practice, many credit institutions face the cold-start problem: they lack enough historical data on new borrowers, or their internal datasets are too small to train modern machine learning models effectively. At the same time, large public datasets are available, but they often differ from the target dataset in both feature structure and customer distribution. When a model is trained directly on Dataset A and then applied to the Dataset B, performance typically falls short of a model trained natively on the target domain - a phenomenon known as negative transfer (Pan & Yang, 2010).

What makes this scenario particularly challenging lies not only in the distributional shift between two credit markets, but also in the fact that the two datasets do not share the same feature space: Lending Club describes borrowing behaviour at the loan level (loan amount, interest rate, FICO score, loan purpose, etc.), whereas the Taiwan Default of Credit Card Clients dataset captures monthly credit card spending behaviour (payment history, bill amount, demographic information, etc.). This is no longer a standard transfer learning problem - where both domains share the same feature types - but rather a situation in which both the dimensionality of the feature space and the data distribution diverge between source and target.

The problem of Heterogeneous Domain Adaptation is formulated precisely to address this situation: leveraging knowledge from a data-rich source domain to a data-scarce target domain, when the two domains differ in both feature space dimensionality and data distribution.

1.2.1. Heterogeneous Domain Adaptation

Heterogeneous Domain Adaptation (HDA) is a special case of transfer learning - more specifically, a sub-branch of domain adaptation. Transfer learning is the broadest framework: leveraging knowledge acquired from one task or data domain to improve learning on another. Within it, domain adaptation addresses a more constrained situation: the task on the source and target is identical (same label types, same prediction objective), but the data distributions differ - $P(X_s) \neq P(X_t)$.

Domain adaptation is further divided into two branches based on feature space structure. Homogeneous domain adaptation assumes that both domains share the same feature space - that is $d_s = d_t$ và $P(X_s) = P(X_t)$ - with only the distribution shifted. Heterogeneous domain adaptation relaxes this assumption: the two domains may have entirely different feature spaces, $d_s \neq d_t$, rendering direct comparison or alignment infeasible.

HDA must therefore simultaneously resolve two problems: structural heterogeneity across feature spaces and distributional shift. Most classical domain adaptation methods such as MMD and CORAL are designed for the homogeneous case and cannot be directly applied when $d_s \neq d_t$ - which is precisely why HFA and DANN with domain-specific encoders were developed to fill this gap.

According to Yang et al. (2020), the formal problem statement of transfer learning revolves around four questions: what to transfer, from where to where, when to transfer, and how to transfer.

1.2.2. What to transfer

Before designing any mechanism, the first question to answer is: what knowledge in the source domain can be meaningfully extracted and transferred to the target? Yang (2020, §2.3.1) categorizes transferable knowledge into four types: (1) instances, (2) feature representations, (3) parameters, and (4) relational knowledge. Not every type is viable in every setting - the choice must follow from the specific characteristics of the dataset pair.

In the context of this thesis, two of the four types are ruled out by structural constraints. Instance-based transfer methods such as TrAdaBoost (Dai et al., 2007) assume that the source and target share the same feature space - a condition that is not satisfied when $d_s \neq d_t$. Parameter-based methods such as fine-tuning from an ImageNet-pretrained model (Yosinski et al., 2014) require a shared network architecture, whereas tabular data with heterogeneous feature structures has no naturally shared architecture. Relational knowledge, on the other hand, requires a relational graph between entities - one that is absent in standard tabular credit data.

This leads to a natural choice for the problem: the object of transfer is featuring representation. Rather than transferring raw data or model parameters directly, both methods in this thesis learn to map data from two heterogeneous feature spaces into a common, comparable representation. Specifically, Heterogeneous Feature Augmentation (HFA) learns two linear thesissions to bring the source and target domains into a shared subspace, while Domain-Adversarial Neural Network (DANN) learns domain-invariant representations through adversarial training against a domain classifier. This representation-space transfer strategy is precisely what Yang (2020, §6.1) identifies as the central approach for the heterogeneous domain adaptation setting.

1.2.3. Where to transfer

A data domain is defined as $\mathcal{D} = \{\mathcal{X}, P(X)\}$, where $\mathcal{X}$ is the feature space and $P(X)$ is the marginal distribution over it. A task is defined as $\mathcal{T} = \{\mathcal{Y}, f(\cdot)\}$, where $\mathcal{Y}$ is the output space and $f(\cdot)$ is the function to be learned.

In this problem, the source domain has $\mathcal{X}_S = \mathbb{R}^{d_S}$ with d_S features after Lending Club preprocessing; $P(X_S)$ is the distribution of borrowers in the US market over the period 2007–2018. The target domain has $\mathcal{X}_T = \mathbb{R}^{d_T}$ with d_T features after TDCCC preprocessing; $P(X_T)$ is the distribution of credit card customers in Taiwan in 2005. Both tasks share $\mathcal{Y} = \{0,1\}$, where label 1 denotes default - semantically aligned, even though the operational definition of default differs between the two domains.

Since $d_S \neq d_T$ (different dimensionality) and $P(X_S) \neq P(X_T)$ (different distributions), this constitutes a Heterogeneous Domain Adaptation problem under the formal definition of Yang (2020, §6.1).

1.2.4. When to transfer

Yang (2020, §2.3.3) cautions that transfer is not always beneficial. Following Rosenstein, Marx, Kaelbling, and Dietterich (2005), transfer yields value only when three conditions are simultaneously satisfied: (i) the source and target domains share a related underlying structure, (ii) the target domain has limited data or labels, and (iii) the transfer method is suited to the type of heterogeneity between the two domains.

Applied to this thesis, the three conditions are met in turn. Both Lending Club and Taiwan Default of Credit Card Clients are credit datasets sharing common concepts such as income, debt-to-income ratio, repayment history, and credit limit - indicating a related underlying structure. Taiwan Default of Credit Card Clients contains 30000 samples - sufficient for modern models but not abundant - whereas Lending Club originally comprises over 2 million records, reduced to 300000 via stratified sampling. Since $d_s \neq d_t$, homogeneous domain adaptation methods (such as Maximum Mean Discrepancy - Gretton et al., 2012) are inapplicable; HFA and DANN with domain-specific encoders are the methods specialized for HDA.

1.2.5. How to transfer

Having identified the knowledge to transfer, the source–target domain pair, and the conditions under which transfer is beneficial, the final question is the selection and implementation of the transfer mechanism. Yang (2020, §2.3.4) categorizes the main mechanisms as: instance reweighting, feature mapping, adversarial alignment, and generative modeling. This thesis implements two mechanisms from distinct methodological traditions, enabling direct comparison in both methodology and performance.

HFA belongs to the feature mapping family. Rather than learning an unconstrained mapping, HFA learns two linear thesions to bring the source and target into a shared subspace W_s and W_t , then augments each representation by concatenating three feature blocks: the shared projection, the domain's original features, and a zero-padding block - producing symmetric augmented feature vectors across both domains. The entire procedure is optimized jointly with a Support Vector Machine classifier via alternating optimization. By design, HFA is a shallow method with a locally convex objective and requires no deep network architecture.

DANN belongs to the adversarial alignment family. Rather than seeking an explicit mapping, DANN learns domain-invariant representations through an adversarial training mechanism: a Gradient Reversal Layer compels the feature extractor to produce representations from which the domain classifier cannot distinguish the domain of origin, while retaining sufficient information for label prediction. DANN is a deep method with a non-convex objective, requiring careful balance between two competing training objectives.

1.2.6. Negative Transfer Problem

Negative transfer refers to the phenomenon in which applying transfer learning degrades performance on the target domain relative to training exclusively on that domain. Following Pan and Yang (2010, §3) and Wang, Dai, Póczos, and Carbonell (2019), negative transfer occurs when:

$$\varepsilon_T(\text{transfer}) > \varepsilon_T(\text{baseline})$$

where $\varepsilon_T(\text{transfer})$ is the error on the target domain after applying the transfer learning method and $\varepsilon_T(\text{baseline})$ is the error when training solely on the target domain.

Wang et al. (2019) highlight three main reasons for negative transfer. First, when the source and target domains diverge too much, there is no meaningful shared representation. Second, the chosen model class may lack transferability-what works well on the source can be too specialized to generalize to the target. Third, an extreme imbalance in dataset sizes can cause the source signal to overwhelm the target; when $|D_S| \gg |D_T|$, the model tends to ignore the target distribution.

In the context of this thesis, the risk of negative transfer is significant and must be carefully managed. The Taiwan Default of Credit Card Clients dataset (Taiwan, 2005, revolving credit card product) and the Lending Club dataset (United States, 2007–2018, fixed-term loan product) differ across multiple dimensions: market, time period, credit product type, currency, and the specific operational definition of default. The label distributions also diverge substantially: while Taiwan Default of Credit Card Clients has a default rate of approximately 22%, Lending Club's charged-off rate sits around 14% after stratified sampling. According to Zhao, Combes, Zhang, and Gordon (2019), label shift causes the H-divergence upper bound to become non-tight - adversarial alignment may inadvertently distort the decision boundary on the target domain. The source-to-target size ratio is 12.5:1 against the 24000-sample Taiwan Default of Credit Card Clients training pool, equivalent to 10:1 against the full 30000-sample Taiwan Default of Credit Card Clients - further constitutes a significant source dominance risk.

The experimental design therefore explicitly tests for negative transfer through a 3-case ablation protocol. Preventive measures are also built into both methods: HFA incorporates Frobenius norm regularization, while DANN uses a slow-start λ schedule, class ratio balancing, and gradient clipping.

1.3. Research Questions

This thesis is designed around four exploratory research questions. Rather than assuming fixed outcomes, each question frames an experiment to test how heterogeneous domain adaptation (HDA) might behave in the context of tabular credit scoring.

Question 1: To what extent can transfer learning (implemented through HFA or DANN techniques) improve credit scoring performance on the Taiwan Default of Credit Card Clients dataset when Lending Club is used as the source domain, compared to a baseline trained only on the target?

Question 2: Which of the two approaches, HFA or DANN, shows greater performance for solving credit scoring tasks, and under what conditions do their strengths or weaknesses become most apparent?

Question 3: How do specific design choices shape the effectiveness of transfer learning, as revealed through the ablation experiments?

Question 4: What risks of negative transfer emerge when applying HDA across domains with different markets, products, and label distributions, and how can experimental protocols detect and mitigate these risks?

Chapter 2. Literature Review

2.1. Overview of Transfer Learning

2.1.1. Definition and Taxonomy

Pan and Yang (2010) published the first survey that formally organized the field of transfer learning. They classified approaches along three dimensions: (i) whether the source and target domains are identical, (ii) whether the source and target tasks are identical, and (iii) whether the target domain is labeled. The resulting taxonomy defines four main classes of transfer learning: inductive, transductive, unsupervised, and negative. Weiss, Khoshgoftaar, and Wang (2016) later extended this taxonomy to include deep learning and neural network–based domain adaptation methods.

The textbook by Yang, Zhang, Dai, and Pan (2020) is one of the most authoritative references currently available. Spanning 470 pages, it covers classical methods, deep transfer learning, reinforcement transfer learning, and lifelong learning. This work serves as the main reference for the present thesis, with particular emphasis on Chapter 6 (HFA) and Chapter 8 (DANN).

2.1.2. Domain Adaptation and H-divergence Theory

The theoretical foundations of domain adaptation were first laid out by Ben-David, Blitzer, Crammer, and Pereira (2007), and later expanded in Ben-David, Blitzer, Crammer, Kulesza, Pereira, and Vaughan (2010). Theorem 2 in Ben-David et al. (2010) provides the following upper bound on target domain risk:

$$\varepsilon_T(h) \leq \varepsilon_S(h) + \frac{1}{2}d_{\mathcal{H}\Delta\mathcal{H}}(\mathcal{D}_S, \mathcal{D}_T) + \lambda^*,$$

where $d_{\mathcal{H}\Delta\mathcal{H}}(\mathcal{D}_S, \mathcal{D}_T)$: the H-divergence between source and target distributions;

$\lambda^* = \min_{h \in \mathcal{H}} [\varepsilon_S(h) + \varepsilon_T(h)]$: the combined error of the ideal joint hypothesis.

The core insights is that if we can reduce the distributional gap between source and target in the representation space (without harming predictive accuracy on the source), we also reduce risk on the target. This result provides the mathematical motivation behind modern domain adaptation methods, including DANN.

Mansour, Mohri, and Rostamizadeh (2009) later introduced the discrepancy distance as a more flexible alternative to H-divergence, particularly when the loss function is not binary. Another line of work builds on Maximum Mean Discrepancy (MMD) (Gretton et al., 2012). Tzeng, Hoffman, Zhang, Saenko, and Darrell (2014) embedded MMD into neural networks through Deep Domain Confusion, while Long, Cao, Wang, and Jordan (2015) generalized this approach into Deep Adaptation Networks. MMD-based methods offer smooth gradients and stable convergence without adversarial training, but they face challenges: kernel selection is non-trivial, and performance often degrades with small batch sizes.

2.1.3. Heterogeneous Domain Adaptation

When the source and target domains differ in feature dimensionality, standard domain adaptation methods cannot be applied directly. The literature offers two main approaches to address this problems.

The first approach is common subspace learning. Wang and Mahadevan (2011) introduced Domain Adaptation via Manifold Alignment using Laplacian eigenmaps. Duan, Xu, and Tsang (2012) laid the groundwork for Heterogeneous Feature Augmentation (HFA), which learns two thesisions that map both domains into a shared subspace, followed by SVM classification on the augmented features. Yang et al. (2020, Chapter 6) later formalized this into a convex optimization framework with Frobenius norm constraints (Equation 6.8).

The second approach relies on deep feature mapping. Tsai, Yeh, and Wang (2016) proposed the Cross-Domain Cross-Modality Transfer Network, combining autoencoders with adversarial alignment. Chen, Hsu, Tsai, Wang, and Chen (2016) introduced Transfer Neural Trees to handle mixed feature types. In general, deep learning methods are more flexible than shallow ones but demand significantly larger datasets.

It is important to note that HFA and DANN represent fundamentally different strategies for heterogeneous domain adaptation: HFA is shallow and convex, offering closed-form theoretical guarantees, while DANN is deep and non-convex, driven primarily by empirical performance. Comparing these two families on the same dataset provides valuable experimental insight.

2.1.4. Adversarial Domain Adaptation

Adversarial domain adaptation builds on the GAN framework introduced by Goodfellow et al. (2014), where two networks compete: an encoder (acting as a generator) tries to fool a discriminator, while the discriminator attempts to distinguish between domains. Ganin and Lempitsky (2015) were the first to apply this idea to domain adaptation, proposing the Domain-Adversarial Neural Network (DANN) with the Gradient Reversal Layer as its central innovation. Ganin et al. (2016) later provided a full account of the architecture, including the λ schedule, learning rate policy, and experiments across six benchmark datasets (such as MNIST→MNIST-M and Office).

Tzeng, Hoffman, Saenko, and Darrell (2017) introduced Adversarial Discriminative Domain Adaptation (ADDA), which uses separate encoders without shared weights-well suited for cases where the source model is pretrained and the target encoder is fine-tuned afterward. Long, Cao, Wang, and Jordan (2018) proposed the Conditional Domain Adaptation Network (CDAN), which incorporates class information into the discriminator via an outer product, achieving state-of-the-art results on several image benchmarks. Saito, Watanabe, Ushiku, and Harada (2018) offered an alternative alignment criterion based on maximum classifier discrepancy.

An important theoretical come from Zhao, Combes, Zhang, and Gordon (2019). They demonstrated that minimizing the empirical adversarial loss does not guarantee minimization of the true H-divergence bound, especially when label distributions differ across domains, $P_S(Y) \neq P_T(Y)$. This is precisely why the thesis must carefully test for negative transfer when $P_S(Y) \neq P_T(Y)$ and $d_S \neq d_T$.

2.2. Overview of Credit Scoring

2.2.1. Classical Statistical Era (1940–1990)

The roots of credit scoring lie in classical statistics. Durand (1941) was the first to apply statistical analysis systematically to consumer credit. Beaver (1966) advanced the field by showing how financial ratios could predict corporate failure, laying the groundwork for corporate credit scoring. Altman (1968) then introduced the Z-score model, a linear combination of five ratios that achieved about 95% accuracy in predicting bankruptcy among U.S. firms. Eisenbeis (1977) criticized the reliance on Gaussian assumptions in linear discriminant analysis and argued for logistic regression. Logistic regression quickly became the industry standard because of its interpretability and its alignment with regulatory requirements.

2.2.2. Classical Machine Learning Era (1990–2010)

By the 1990s, machine learning methods began to reshape credit scoring. Henley (1995) was among the first to apply k-Nearest Neighbors and neural networks to this domain. Baesens, Van Gestel, Viaene, Stepanova, Suykens, and Vanthienen (2003) carried out a large benchmark study, testing 17 algorithms across 8 datasets, and established support vector machines with RBF kernels as a strong baseline. Yeh and Lien (2009) introduced the well-known TDCCC dataset and compared six classification methods on it. Around the same time, Hand (2009) raised the “measure problem”: he showed that the choice of evaluation metric (AUC, KS, accuracy, etc.) can change the ranking of models, highlighting the importance of reporting multiple metrics rather than relying on a single one.

2.2.3. Modern Machine Learning and Deep Learning (2010 to present)

The 2010s marked the era of large-scale data and deep learning. Khandani, Kim, and Lo (2010) applied boosting and CART to transaction data from a major U.S. bank, setting an early benchmark for big-data credit scoring. Lessmann et al. (2015) later conducted the largest benchmark to date, comparing 41 classifiers across 8 datasets. They concluded that ensemble methods such as Random Forest and Gradient Boosting consistently improve AUC by 1–3 percentage points over logistic regression.

Deep learning entered the field soon after. Babaev, Savchenko, Tuzhilin, and Umerenkov (2019) introduced E.T.-RNN, applying recurrent neural networks to transaction data at Sberbank, and established a reference point for transfer learning in fintech. More recently, Borisov, Leemann, Seßler, Haug, Pawelczyk, and Kasneci (2024) showed that on tabular data, gradient boosting methods (XGBoost, LightGBM, CatBoost) generally match or outperform purely deep models. Deep approaches tend to gain an edge only when combined with self-supervised pretraining or transfer learning - precisely the setting explored in this thesis.

2.3. Overview of applying Transfer Learning for Credit Scoring

2.3.1. Homogeneous Transfer Learning

Tsai and Wu (2008) were among the earliest to address multi-source learning for credit scoring, using an ensemble drawn from multiple bank datasets sharing the same feature schema. Tobback

and Martens (2017) presented a transfer learning framework for telecom-based credit scoring, leveraging call detail records to score customers with no credit history.

Bracke, Datta, Jung, and Sen (2019) produced one of the most authoritative studies from a regulatory perspective, applying GBM to Lending Club data with a detailed explainability analysis. The authors found that machine learning can improve AUC over logistic regression, but that the interpretability trade-off warrants careful consideration for regulatory acceptance.

2.3.2. Heterogeneous Transfer Learning

Liu et al. (2018) applied deep transfer learning to credit scoring via feature mapping through a Deep Belief Network. However, relatively few studies have focused specifically on the heterogeneous setting ($d_s \neq d_t$) for credit scoring - the gap this thesis aims to address.

2.3.3. Adversarial and GAN-based Methods in Credit Scoring

Engelmann and Lessmann (2021) applied conditional Wasserstein GAN (cWGAN) to imbalanced credit scoring – not a domain adaptation method per se, but analogous in adversarial design. Xu and Zhang (2025) proposed an explainable domain adaptation learning framework for credit scoring in internet finance, combining adversarial transfer learning with ensemble fusion to improve predictive accuracy. Direct application of DANN to heterogeneous credit scoring remains a relatively unexplored direction; the majority of DANN literature focuses on image domains (MNIST, Office, VisDA). The implementation in this thesis is among the few HeteroDANN applications to tabular credit data.

2.4. Negative Transfer

Rosenstein, Marx, Kaelbling, and Dietterich (2005) were the first to publicly question whether transfer is always beneficial, providing clear examples of negative transfer when source and target are sufficiently dissimilar. Pan and Yang (2010, §3) formalized the definition and proposed avoidance strategies: careful source selection and instance reweighting. Wang, Dai, Póczos, and Carbonell (2019) offered the most thorough investigation of negative transfer to date, proposing a discriminator-based safety check prior to transfer.

In the credit scoring context, direct evidence of negative transfer comes from: when the source (consumer loans) and target (small business loans) diverge substantially in median loan amount, transfer reduces AUC - a direct warning for the present thesis.

2.5. Research Gap

The literature review reveals three principal gaps.

(i) Research on heterogeneous domain adaptation (HDA) in credit scoring is still limited. Most transfer learning studies in this area assume homogeneous domains, where source and target share the same feature space.

(ii) The two major families of HDA methods-shallow convex approaches and deep adversarial approaches-are usually benchmarked against each other in image or text applications. They are rarely compared side-by-side on tabular financial data, which presents unique challenges.

(iii) There is a shortage of rigorous benchmarks for HDA in credit scoring. Existing studies often lack structured experimental protocols such as three-case ablation, five-fold cross-validation, multi-metric evaluation, and reproducibility guarantees.

This thesis is designed to address all three gaps simultaneously. It provides full implementations of both HFA and DANN, explicitly tests for negative transfer through ablation experiments, and ensures reproducibility by releasing source code together with a theoretical reference document.

Chapter 3. Datasets and Domain Analysis

3.1. Target Domain - Taiwan Default of Credit Card Clients (TDCCC)

The Taiwan Default of Credit Card Clients dataset was introduced by Yeh and Lien (2009) and made publicly available on the UCI Machine Learning Repository. It contains records for 30000 credit card customers in Taiwan, collected from April to September 2005. The target variable is the probability of default in the following month, with a default rate of 22.12% - consistent with the figure reported by Yeh and Lien (2009).

3.1.1. Feature description

The dataset comprises 30,000 records $\times$ 25 columns (including ID and label). Source: Yeh & Lien (2009), UCI Repository.

Table 1: Target domain - Taiwan Default of Credit Card Clients feature description

Group	Representative features	Description
Continuous (14)	LIMIT_BAL, AGE, BILL_AMT1– BILL_AMT6, PAY_AMT1– PAY_AMT6	Amount of given credit (NT\$); age in years; monthly bill statement amounts (Apr–Sep 2005); monthly payment amounts (Apr–Sep 2005)
Nominal (3)	SEX, EDUCATION, MARRIAGE	Gender (1 = male, 2 = female); education level (1 = graduate school, 2 = university, 3 = high school, 4 = others); marital status (1 = married, 2 = single, 3 = others)
Ordinal (6)	PAY_0, PAY_2, PAY_3, PAY_4, PAY_5, PAY_6	Repayment status for Sep–Apr 2005; scale: –1 = pay duly, 1 = one-month delay, ..., 8 = eight-month delay, 9 = nine months and above
Identifier (1)	ID	Row identifier (excluded from modelling)
Target label (1)	default.payment.next.month	Binary (1 = default, 0 = non-default)

3.1.2. Taiwan Default of Credit Card Clients preprocessing

The preprocessing pipeline for TDCCC was implemented in strict adherence to the original codebook of Yeh (2009) and consists of four core steps: normalization of the EDUCATION variable, normalization of the MARRIAGE variable, clipping of the PAY_X repayment history variables, and explicit type casting to ensure consistency with the training pipeline.

Step 1 - EDUCATION normalization. The official codebook (Yeh 2009, §3) defines four valid codes: 1 = Graduate school, 2 = University, 3 = High school, 4 = Others. However, the raw data contains undocumented codes {0, 5, 6} that are absent from the original codebook. Following the UCI dataset metadata recommendation, all such codes are remapped to 4 ("Others"). This strategy avoids introducing new category levels with no training observations while preserving all data rows - a standard practice in preprocessing under label scarcity (Géron 2019, Ch. 2). After this step, EDUCATION is guaranteed to take values exclusively in {1, 2, 3, 4}.

Step 2 - MARRIAGE normalization. The same logic is applied to MARRIAGE. The codebook defines three valid codes: 1 = Married, 2 = Single, 3 = Others. Code 0 (undocumented) appears in the raw data and is remapped to 3 ("Others"). This treatment is consistent with the rationale applied to EDUCATION: the label space is not expanded beyond the definitions in the original documentation, and the full dataset is retained. After this step, MARRIAGE is guaranteed to take values in {1, 2, 3}.

Step 3 - PAY_X clipping. The codebook (Yeh 2009) provides complete semantic definitions for the valid range: -2 denotes no consumption, -1 denotes payment in full, 0 denotes use of revolving credit, and values 1 through 8 denote the corresponding number of months in arrears. In practice, some rows contain out-of-range values due to data entry errors. Following Yeh (2009, §4.1), clipping to [-2, 8] is the standard approach for removing noise without information loss.

Step 4 - Type casting. Explicit type casting is applied to ensure compatibility (Géron, 2019). Categorical and ordinal variables - SEX, EDUCATION, MARRIAGE, and all PAY_X columns - are cast to int. Continuous numeric variables - LIMIT_BAL, BILL_AMT1-6, and PAY_AMT1-6 - are cast to float. This explicit separation between the two type groups also serves the subsequent step: constructing feature lists by role (numeric, nominal, ordinal) for input into the imputation, standardisation, and classification pipeline without data leakage.

3.2. Source Domain - Lending Club (LC)

The Lending Club Loan Dataset (United States, 2007–2018) contains 2260668 records with 145 raw features. It is the largest publicly available peer-to-peer lending dataset, reflecting the US consumer credit market over a period that encompasses the 2008 financial crisis.

3.2.1. Feature description

The dataset comprises 2,260,668 records $\times$ 145 columns. Source: Lending Club public loan data, 2007–2018.

Table 2: Source domain - Lending Club feature description

Group	Representative features	Description
Continuous (~ 80)	loan_amnt, funded_amnt, funded_amnt_inv, int_rate, installment, annual_inc, dti, delinq_2yrs, inq_last_6mths, open_acc, pub_rec,	Loan amount, funded amount, interest rate, monthly instalment, annual income, debt-to-income ratio, delinquencies, inquiries, account

Group	Representative features	Description
	revol_bal, revol_util, total_acc, fico_range_low, fico_range_high, tot_cur_bal, total_rev_hi_lim, acc_open_past_24mths, avg_cur_bal, mo_sin_old_il_acct, mort_acc, num_actv_bc_tl, num_bc_sats, num_il_tl, num_op_rev_tl, num_rev_accts, num_sats, pct_tl_nvr_dlq, percent_bc_gt_75, tot_hi_cred_lim, total_bal_ex_mort, total_bc_limit, total_il_high_credit_limit, ...	counts, revolving balance and utilisation, FICO score range, and various credit bureau aggregates
Nominal (7)	home_ownership, verification_status, purpose, title, addr_state, application_type, disbursement_method	Home ownership status (RENT/OWN/MORTGAGE), income verification status, loan purpose (debt_consolidation, credit_card, ...), borrower state, application type (Individual/Joint), disbursement method
Ordinal (3)	grade, sub_grade, emp_length	LC internal rating A–G and subgrade A1–G5 (based on FICO and credit profile); employment length (< 1 year to 10+ years)
Binary (5)	term, pymnt_plan, initial_list_status, hardship_flag, debt_settlement_flag	Loan term (36/60 months), payment plan (y/n), listing status (w/f), hardship programme flag, debt settlement flag
Date (5)	issue_d, earliest_cr_line, last_pymnt_d, last_credit_pull_d, next_pymnt_d	Loan issue date, earliest credit line date, last payment date, last credit pull date, next payment date
Text (5)	emp_title, title, desc, url, zip_code	Job title, loan title, borrower description, loan URL, zip code (first 3 digits only)
Post-loan / leakage (~ 30)	out_prncp, total_pymnt, total_rec_prncp, total_rec_int, total_rec_late_fee, recoveries, collection_recovery_fee, last_pymnt_amnt, chargeoff_within_12_mths, debt_settlement_flag_date,	Variables populated only after loan disbursement - removed during preprocessing to prevent target leakage

Group	Representative features	Description
	settlement_*,...	
Joint applicant (10)	annual_inc_joint, dti_joint, verification_status_joint, sec_app_*	Joint application and secondary applicant variables - mostly >90% missing, removed
Identifier (3)	id, member_id, policy_code	Row and member identifiers, policy code (excluded from modelling)
Target label (1)	loan_status	Charged Off / Default / Late (31–120 days) → 1; Fully Paid → 0

3.2.2. Lending Club preprocessing

The raw Lending Club dataset comprises 2260668 loan records and 145 feature columns. Before being used as the source domain in transfer learning experiments, it undergoes a preprocessing pipeline whose overall objectives are to eliminate data leakage, standardize data types, and reduce the feature space to a dimensionality appropriate for heterogeneous domain adaptation.

Step 1 - High-missingness column removal. Thirty-eight columns with more than 90% missing values are dropped entirely. These belong predominantly to the hardship_*, settlement_*, and sec_app_* groups, as well as variables associated with joint loan applications - information that appears only in distressed records and is therefore unrepresentative for training. The dataset after this step is of shape (2260668, 107).

Step 2 - Post-origination leakage removal. Twenty-three columns containing post-loan information are removed to prevent data leakage, including out_prncp, total_pymnt, recoveries, last_pymnt_d, and similar variables. By construction, these columns are only available after loan disbursement and therefore cannot be used to predict risk at the point of credit decision - a violation of the causal ordering principle in credit modelling (Siddiqi, 2012). The dataset after this step is of shape (2260668, 84).

Step 3 - Label construction. The target variable is derived from loan_status using an explicit binary mapping: the statuses "Charged Off", "Default", and "Late (31–120 days)" are mapped to $y = 1$ (default); "Fully Paid" is mapped to $y = 0$. Intermediate statuses such as "Current" and "In Grace Period" are excluded from training, as their binary labels are undefined at the time of observation - a standard practice in credit scoring label construction (Thomas et al., 2002). After this filtering step, 1325535 records remain, with a default rate of 21.39%.

Step 4 - Imputation. A tiered imputation strategy is applied based on missingness rate. For columns with fewer than 10% missing values, median (for numeric variables) or mode (for categorical variables) imputation is used to preserve the original distribution. For columns with missingness between 10% and 90%, median imputation is applied to numeric variables, while categorical variables receive the sentinel value "unknown". This sentinel explicitly encodes the state of data absence as a distinct value that tree-based models such as XGBoost and Random

Forest can learn to split on directly, rather than treating it as implicitly missing (Chen & Guestrin, 2016).

Step 5 - Encoding. Three encoding schemes are applied according to the nature of each variable group. Binary encoding is applied to five binary columns: term (60 months $\rightarrow$ 1), pymnt_plan, initial_list_status, application_type, and disbursement_method. Label encoding is applied to three ordinal variables: grade (A = 1 to G = 7), sub_grade (1–35 in order from A1 to G5), and emp_length (0–10 years; unknown values imputed with the median). One-hot encoding with the first dummy dropped is applied to four nominal variables: home_ownership, verification_status, purpose, and region - where addr_state is first consolidated into the four standard US geographic regions (Northeast, Midwest, South, West) before encoding. Dropping the first dummy column per group avoids perfect multicollinearity in regression-based models (Géron, 2019).

Step 6 - Cleanup and final shape. Remaining object-type columns (typically issue_d, last_credit_pull_d) are dropped, and any residual NaN values arising from incomplete label encoding mappings are median-imputed. The dataset at this stage has shape (1325535, 121).

Step 7 - Stratified subsampling. Since the LC dataset far exceeds the scale required for the experiments, stratified subsampling is performed using stratified shuffle splitting (with a fixed random seed) to select 300000 records, ensuring the default rate in the subsample is preserved at 21.39%.

All experiments in this study - including both HFA and DANN - consistently use these 300,000 Lending Club records as the source domain. Following the complete pipeline, the source domain feature space reaches $d_S = 120$ (Lending Club), while the target domain retains $d_T = 23$ (TDCCC) - constituting a canonical heterogeneous domain adaptation setting where $d_S \neq d_T$.

3.2.3. Lending Club Feature selection cases

Case 1 - Full feature set

After preprocessing, the Case 1 feature set comprises 300000 observations and 121 input features (1 target variable and 120 others).

Case 2 - Selected feature set

Feature selection in Case 2 is grounded in the 5 C's of Credit framework (Altman & Saunders, 1997), a widely recognized credit evaluation standard in quantitative finance. This approach ensures that the retained features exhibit semantic alignment between the source domain (Lending Club, LC) and the target domain (Taiwan Default of Credit Card Clients, TDCCC) - a prerequisite for effective transfer learning.

Table 3: Lending Club's selected feature set

Criterion	LC features	TDCCC features
Character (credit history)	delinq_2yrs, grade, sub_grade, cr_history_yr	PAY_0 – PAY_6 (repayment history)

Criterion	LC features	TDCCC features
Capacity (repayment ability)	annual_inc, dti, emp_length, installment	LIMIT_BAL, PAY_AMT1 – PAY_AMT6
Capital (credit exposure)	loan_amnt, revol_util, open_acc, total_acc	BILL_AMT1 – BILL_AMT6, LIMIT_BAL
Collateral (security / demographics)	home_ownership_* (one-hot)	MARRIAGE
Conditions (loan context)	int_rate, term, purpose_*, verification_status_*	EDUCATION

The decision to discard approximately 60–70% of LC's original features in Case 2 was made on two grounds. First, variables related to platform policy, collections, recovery, and various internal operational metrics of the P2P platform have no counterpart in TDCCC - a traditional Taiwanese bank dataset - and retaining them would introduce severe feature space asymmetry. Second, following Pan and Yang (2010, §3.3), when the domain gap between source and target is large, restricting the feature space to a semantically aligned subset can help mitigate negative transfer.

The reduction from 121 features in Case 1 to 37 in Case 2 represents a compression of approximately 69%, reflecting a deliberate strategy of prioritizing semantic quality over feature quantity under high domain gap conditions.

3.2.4. Stratified subsampling of Lending Club

The decision to subsample down to 300000 records rests on three considerations. First, computational feasibility: HFA requires the source and target to be jointly processed within the same SVM dual problem; a kernel matrix over 2.26 million samples would require on the order of 2.26×10^{12} entries - infeasible in memory. Second, diminishing returns: according to Halevy, Norvig, and Pereira (2009), learning curves typically saturate beyond 100000 samples, making 300000 more than sufficient for stable interpolation. Third, the resulting source-to-target ratio of $|D_S|/|D_T| = 10$ against the full TDCCC (12.5 against the training pool of 24,000 samples) strikes a balance between source richness and the risk of source dominance.

3.3. Domain Comparison

Table 4: Domain comparison

Criterion	TDCCC	LC
Country	Taiwan	United States
Period	April–September 2005	2007–2018
Product	Credit card (revolving)	Term loan (36/60 months)

Criterion	TDCCC	LC
Sample size	30000	~2.26M (subsampled to 300000)
Number of features	$d_T = 23$	$d_S = 120$ (Case 1); $d_S = 37$ (Case 2)
Default rate	~22%	~14%
Currency	TWD	USD
Default definition	Missed payment in the following month	Charged Off / Default

The degree of heterogeneity between the two domains can be analyzed along four dimensions.

Feature space heterogeneity. $\mathcal{X}_S \neq \mathcal{X}_T$ and $d_S \neq d_T$ - heterogeneous by definition under Yang (2020, §6.1).

Label shift. $P_S(Y) \neq P_T(Y)$ (~14% vs. ~22% default rate). Following Zhao et al. (2019, ICML), this divergence poses a substantial risk for adversarial domain adaptation: aligning marginal feature distributions $P(X)$ does not guarantee alignment of the conditional $P(Y | X)$, and the H-divergence bound may no longer be tight.

Covariate shift. The two domains differ in currency, income scale (Taiwan median income in 2005 was approximately \$15,000 USD versus a US median of approximately \$65,000 in 2017), and credit culture - diverging in both scale and distributional shape.

Concept shift. $P_S(Y | X) \neq P_T(Y | X)$: the definition of default differs (one missed monthly payment versus charge-off); the prediction horizon also differs - the same input values may carry different labels across the two domains.

Three distinct negative transfer risks are identified. The first arises from label shift: if DANN is not carefully constrained, the encoder will learn to align marginal distributions while inadvertently distorting the class boundary on the target. This is mitigated by increasing `target_cls_weight` to amplify the target classification signal. The second risk stems from source dominance: with a 10:1 source-to-target ratio, gradients from the source will overwhelm those from the target. Mitigation combines target sample weighting with a slow-start λ schedule, deferring full adversarial pressure rather than applying it from the first iteration. The third risk arises from conceptual mismatch between loan and credit card products: this cannot be eliminated by design and must instead be diagnosed empirically - specifically by comparing Case 3 against Case 1 in the ablation protocol. If Case 3 fails to outperform Case 1, the source–target pair is deemed unsuitable for transfer.

Chapter 4. Research Methodology

4.1. Baseline models

4.1.1. Models' selection

Three models are selected to represent three prevailing paradigms in credit scoring (Lessmann et al., 2015): logistic regression represents classical linear classification - the industry standard in credit scoring since Cox (1958); Random Forest represents bagging-based ensemble learning; and XGBoost represents gradient boosting - the state of the art on many tabular benchmarks (Chen & Guestrin, 2016, KDD).

4.1.2. Data splitting

The data are split at an 80/20 ratio using stratified sampling with `random_state=42`, preserving the default rate across both sets:

- Train pool: 24000 samples (default rate = 0.2212)
- Hold-out test: 6000 samples (default rate = 0.2212)

The class distribution on the training set is 18691 non-default versus 5309 default, yielding `scale_pos_weight = 3.521` ($= 18691 / 5309$). This value is passed directly to XGBoost to compensate for class imbalance.

4.1.3. Baseline experimental design

Each model is wrapped in a Pipeline. The `StandardScaler` is fitted exclusively on the training fold during cross-validation, and exclusively on the train pool during hold-out evaluation, preventing data leakage - no information from the test set is used to normalize the data.

4.1.3.1. Logistic Regression

Table 5: Baseline method - Logistic Regression's parameters

Hyperparameter	Value	Rationale
C	0.1	Stronger regularization than the default (1.0) to avoid overfitting on 23 features; a smaller C corresponds to a stronger L2 penalty, appropriate when the feature count is low
max_iter	1000	Ensures convergence for the L-BFGS solver
class_weight	"balanced"	Automatically adjusts class weights inversely proportional to class frequencies: $w_k = n / (K \cdot n_k)$
solver	"lbfgs"	Quasi-Newton method; efficient for L2 penalty and converges faster than SGD on medium-sized datasets

Hyperparameter	Value	Rationale
random_state	42	Reproducibility

4.1.3.2. Random Forest

Table 6: Baseline method - Random Forest's parameters

Hyperparameter	Value	Rationale
n_estimators	200	Sufficient to reduce variance; Breiman (2001) showed that increasing the number of trees does not cause overfitting, though gains diminish beyond ~200 trees
max_depth	8	Limits tree depth to prevent overfitting while allowing sufficient interaction terms
class_weight	"balanced"	Class imbalance correction, analogous to logistic regression
random_state	42	Reproducibility
n_jobs	-1	Full CPU parallelization

4.1.3.3. XGBoost

Table 7: Baseline method - XGBoost's parameters

Hyperparameter	Value	Rationale
n_estimators	200	Matched to Random Forest for fair comparison
max_depth	4	Shallower than Random Forest - in boosting, each tree learns from the residuals of its predecessor, so shallower trees are preferred (Friedman, 2001)
learning_rate	0.05	Low shrinkage rate for better generalization; trade-off: more trees are required, but overfitting is reduced
scale_pos_weight	3.521	$= (\#neg) / (\#pos) = 18,691 / 5,309$; native XGBoost mechanism for class imbalance correction
eval_metric	"logloss"	Appropriate for binary classification; measures calibration quality
random_state	42	Reproducibility

4.1.4. Model evaluation

Evaluation proceeds in two stages.

Stage 1 - 5-fold stratified cross-validation on the train pool (24000 samples): assesses stability and generalization within the training set, with each fold preserving the original default rate.

Stage 2 - Hold-out evaluation on 6000 test samples: assesses performance on entirely unseen data. The classification threshold is selected via KS-optimal criterion on the training set (Youden, 1950):

$$\hat{\tau}^* = \arg \max_{\tau} (\text{Sensitivity}(\tau) - (1 - \text{Specificity}(\tau)))$$

and then applied to the test set - ensuring no leakage from the test set into threshold selection.

4.1.4.1. KS-Optimal Threshold

For each model, the classification threshold is chosen as the KS-optimal threshold (Youden, 1950):

$$\tau^* = \arg \max_{\tau} J(\tau), J(\tau) = \text{Sensitivity}(\tau) + \text{Specificity}(\tau) - 1$$

where $J(\tau)$ is Youden's statistic. Krzanowski and Hand (2009) demonstrate that

$$\max_{\tau} J(\tau) = \max_{\tau} (\text{TPR}(\tau) - \text{FPR}(\tau)) = \text{KS statistic}$$

so the J-optimal threshold coincides with the KS-optimal threshold. The threshold is determined on the training fold and applied to the test fold to avoid threshold overfitting.

4.1.4.2. Metrics

Brier Score (Brier, 1950) measures the mean squared deviation between predicted probabilities and true labels, and is the most important calibration metric alongside AUC:

$$\text{BS} = \frac{1}{N} \sum_{i=1}^N (\hat{p}_i - y_i)^2$$

$\text{BS} \in [0,1]$; lower values are better. $\text{BS} = 0$ corresponds to perfect prediction; $\text{BS} = 0.25$ corresponds to a model that always predicts probability 0.5 (no information). Unlike AUC, which measures only ranking order, Brier Score penalizes heavily when the model is confidently wrong. This is the motivation for applying Platt calibration after HFA - so that Brier Score reflects true probability quality, not merely ranking.

Sensitivity (recall for the positive class) and Specificity (recall for the negative class) are derived from the confusion matrix:

$$\text{Sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

In credit scoring with asymmetric costs, Sensitivity - the true detection rate for defaulters - is of greater importance than Specificity. However, maximizing Sensitivity alone reduces Specificity, increasing false positives and leading to unjustified credit rejections. The KS-optimal threshold balances the two according to Youden's J. Accuracy = $(\text{TP} + \text{TN}) / N$ is reported for completeness but is not used as a primary metric, as class imbalance (22% default in TDCCC) renders it misleading.

AUC-ROC (Fawcett 2006) measures the area under the ROC curve - equivalent to the probability that the model ranks a random positive instance higher than a random negative instance:

$$AUC = \int_0^1 TPR(FPR^{-1}(u)) du = P(\hat{p}(x^+) > \hat{p}(x^-))$$

In the codebase, AUC is computed via the trapezoidal rule over the discrete ROC curve - equivalent to the Wilcoxon–Mann–Whitney statistic (Hand, 2009). AUC is threshold-independent, enabling model comparison prior to threshold selection. In credit scoring, $AUC \geq 0.70$ is generally considered acceptable under Basel III standards (BCBS, 2017, §4.5).

4.2. Heterogeneous Feature Augmentation (HFA) Methodology

4.2.1. HFA theoretical foundation

Heterogeneous Feature Augmentation was introduced by Duan, Xu, and Tsang (2012) and systematized into a convex optimization framework in Yang, Zhang, Dai, and Pan (2020, *Transfer Learning*, Cambridge University Press, Chapter 6, Equation 6.8).

The core idea is that rather than forcing source and target into a shared space - infeasible when $d_s \neq d_t$, HFA transforms each sample into an augmented vector comprising three blocks:

$$\varphi_s(x_s) = \begin{bmatrix} P x_s \\ x_s \\ 0_{d_t} \end{bmatrix} \in \mathbb{R}^{d_c+d_s+d_t}, \quad \varphi_t(x_t) = \begin{bmatrix} Q x_t \\ 0_{d_s} \\ x_t \end{bmatrix} \in \mathbb{R}^{d_c+d_s+d_t},$$

where $P \in \mathbb{R}^{d_c \times d_s}$ and $Q \in \mathbb{R}^{d_c \times d_t}$ are learned linear thesisions. The three blocks serve distinct roles: the first block (Px_s and Qx_t , both of dimension d_c) constitutes the shared subspace, enabling direct comparison between source and target; the second and third blocks retain each domain's original raw features to avoid information loss; and the zero-padding of the opposing domain's block enforces separation between domain-specific information.

After augmentation, all points reside in the same space $\mathbb{R}^{d_c+d_s+d_t}$, allowing a single SVM to be fitted jointly.

This augmentation structure has an important property concerning inner products in the shared space. Consider the inner product between any two points - one from the source, one from the target:

$$\varphi_s^T(x_s) \cdot \varphi_t(x_t) = (Px_s)^T(Qx_t) + x_s^T \cdot 0_{d_t} + 0_{d_s}^T \cdot x_t = x_s^T P^T Q x_t$$

This shows that all cross-domain interaction between a source point and a target point is mediated through $P^T Q$ - that is, through the k -dimensional shared subspace. The raw blocks (x_s and x_t) do not interact directly due to zero-padding, so the SVM can only learn source–target associations through the shared subspace. This guarantees that if P and Q learn a good shared representation capturing semantic similarity, the transferred knowledge is meaningful; if they do not, each domain's raw block still preserves its original information as a fallback.

The subspace dimensionality k balances two competing requirements: a larger k allows the shared space to capture more complex relationships, but increases the number of parameters to learn (P has $d_C \times d_S$ elements, Q has $d_C \times d_T$) and raises the risk of overfitting when $|D_T|$ is small. The notebook selects $k \in \{64\}$, $d_C \approx 2.5 \times d_T$ respectively - ensuring the shared space is sufficiently expressive without exceeding the degrees of freedom available in the target data.

4.2.2. Objective function – Equation 6.8 (Yang, 2020)

Given a mixed training set with labeled source samples $\{(x_i^S, y_i)\}_{i=1}^{n_S}$ and labeled target samples $\{(x_j^T, y_j)\}_{j=1}^{n_T}$ with $y_i \in \{-1, +1\}$, HFA solves:

$$\begin{aligned} \min_{P, Q, w, b, \xi} \quad & \frac{1}{2} \|w\|^2 + \frac{\lambda_P}{2} \|P\|_F^2 + \frac{\lambda_Q}{2} \|Q\|_F^2 + C \sum_{i=1}^{n_S+n_T} \xi_i^2 \\ \text{s.t.} \quad & y_i(w^\top \varphi(x_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \\ & \|P\|_F \leq r_P, \|Q\|_F \leq r_Q. \end{aligned}$$

Each term has a distinct role. The term $\frac{1}{2} \|w\|^2$ is the L2 regularization for the SVM hyperplane (Cortes & Vapnik, 1995). The terms $\frac{\lambda_P}{2} \|P\|_F^2$ and $\frac{\lambda_Q}{2} \|Q\|_F^2$ are soft Frobenius regularization terms, preventing the thesions from growing unboundedly and preserving the interpretability of the raw blocks (Hastie, Tibshirani & Friedman, 2009). The term $C \sum_i \xi_i^2$ is the total squared hinge loss (Chapelle, 2007). The hard constraints $\|P\|_F \leq r_P$ and $\|Q\|_F \leq r_Q$ form a Frobenius ball constraint (Yang, 2020, §6.3.4), guarding against negative transfer by preventing the common subspace block from dominating the raw block

4.2.3. Frobenius projection

Boyd and Vandenberghe (2004) establish that the set $\mathcal{C}_r = \{M: \|M\|_F \leq r\}$ is a closed convex set and that the Euclidean projection onto it has the closed-form solution:

$$\begin{aligned} \Pi_r(M) &= M \cdot \min(1, \frac{r}{\|M\|_F}) \\ \text{proj}_{\mathcal{C}_r}(M) &= \begin{cases} \frac{r}{\|M\|_F} M, & \text{if } \|M\|_F > r, \\ M, & \text{other.} \end{cases} \end{aligned}$$

4.2.4. Johnson–Lindenstrauss (JL) random projection initialization

The thesions P and Q are initialized using Gaussian random matrices following Achlioptas (2003) - a finite-sample instantiation of the Johnson–Lindenstrauss lemma:

$$\mathbf{P}_{ij} \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 1/d_C), \quad \mathbf{Q}_{ij} \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 1/d_C).$$

By the JL theorem,

$$\Pr[(1 - \varepsilon) \|\mathbf{x}\|^2 \leq \|\mathbf{P}\mathbf{x}\|^2 \leq (1 + \varepsilon) \|\mathbf{x}\|^2] \geq 1 - 2\exp(-d_c \varepsilon^2/8),$$

preserves the Euclidean norm of each point with high probability. For $d_c = 64$ and $\varepsilon = 0,5$, the preservation probability exceeds $\geq 0,73$; for $d_c = 128$, this probability remains above $\geq 0,97$. Following initialization, P and Q are immediately projected onto the Frobenius ball to ensure a feasible starting point.

4.2.5. Subgradient computation (P & Q)

The squared-hinge loss

$$\ell(z) = \max(0, 1 - z)^2, \ell'(z) = -2\max(0, 1 - z)$$

is differentiable everywhere. Applying the chain rule to $\ell(y_i w^\top \varphi(x_i))$ and differentiating with respect to P :

$$\frac{\partial \ell_i}{\partial P} = -2C \max(0, 1 - y_i z_i) \cdot y_i \cdot w_c x_{S,i}^\top \forall i \in \mathcal{A}_S, z_i = w^\top \varphi(x_i) + b$$

where $w_c \in \mathbb{R}^{d_c}$ is the portion of w corresponding to the common-subspace block. Analogously for Q :

$$\frac{\partial \ell_i}{\partial Q} = -2C \max(0, 1 - y_i z_i) \cdot y_i \cdot w_c x_{T,i}^\top i \in \mathcal{A}_T$$

The subgradient of the regularizer $\partial(\lambda_P/2 \cdot \|P\|_F^2)/\partial P = \lambda_P P$ (analogously for Q).

The active set $\mathcal{A}$ consists of margin violators $\{i: 1 - y_i z_i > 0\}$ - following Hsieh, Chang, Lin, Keerthi, and Sundararajan (2008, ICML, Algorithm 1). Note that an earlier version of the codebase was missing the coefficient C in the subgradient - a bug corrected in the v3 final file, documented in the #note of the `_grad_PQ` function.

Combining the loss and regularizer gradients, the complete gradient of the objective with respect to P and Q - as implemented in `_grad_PQ()` - is:

$$g_P = -2C \sum_{i \in \mathcal{A}_S} \max(0, 1 - y_i z_i) \cdot y_i \cdot w_c x_{S,i}^\top + \lambda_P P$$

$$g_Q = -2C \sum_{i \in \mathcal{A}_T} \max(0, 1 - y_i z_i) \cdot y_i \cdot w_c x_{T,i}^\top + \lambda_Q Q$$

The first term is the loss gradient, computed only over the active set $\mathcal{A}_S$ or $\mathcal{A}_T$ - the margin violators with $1 - y_i z_i > 0$; the second term is the soft Frobenius regularizer gradient. The projection gradient update is then:

$$P \leftarrow \text{proj}_{c_{rP}}(P - \eta g_P), Q \leftarrow \text{proj}_{c_{rQ}}(Q - \eta g_Q)$$

The coefficient C in the loss term is the SVM regularization parameter. Omitting C causes the gradient to be under-scaled, making P and Q update far too slowly relative to the SVM step.

4.2.6. Block Coordinate Descent

Since the objective in Equation 6.8 is not jointly convex in (P, Q, w) but is convex in each block when the others are held fixed, Block Coordinate Descent is applied (Tseng, 2001, Theorem 4.1).

The algorithm as implemented in `train_hfa` proceeds as follows.

Initialization: $(P_0, Q_0) \sim \mathcal{N}(0, 1/d_c)$, followed by immediate projection onto the Frobenius ball.

Loop for $k = 1, \dots, n_{\text{outer}}$: (i) augment both domains with (P_{k-1}, Q_{k-1}) , then fit LinearSVC to update (w_k, b_k) ; (ii) compute the subgradient $\nabla_{P,Q} L$ at the current point and apply projection gradient steps $P_k \leftarrow P_{k-1} - \eta g_P$, $Q_k \leftarrow Q_{k-1} - \eta g_Q$; (iii) projection (P_k, Q_k) onto the Frobenius ball.

The convergence theorem (Tseng, 2001, Theorem 4.1) guarantees that every accumulation point of the sequence $\{(P_k, Q_k, w_k)\}$ is a stationary point of L , provided three conditions hold: (i) L is continuous in each block and convex in each block when the others are fixed; (ii) the feasible set is a Cartesian product of closed convex sets (Frobenius ball $\times \mathbb{R}^d$); (iii) each sub-step (LinearSVC, projection gradient) guarantees a non-increasing objective value.

4.2.7. Platt probability calibration

The SVM output is a decision score $f(x) = w^\top \varphi(x) + b \in \mathbb{R}$, not a probability. Platt scaling (Platt, 1999) is applied:

$$\hat{P}(y = 1 | x) = \sigma(Af(x) + B) = \frac{1}{1 + \exp(-Af(x) - B)}$$

where (A, B) are estimated by MLE via scikit-learn's LogisticRegression with a single input feature f and target y . Setting $C = 10^6$ (very weak regularization) is equivalent to pure MLE; `class_weight='balanced'` corrects for class imbalance (King & Zeng, 2001).

4.2.8. HFA experimental design

Table 8: HFA method's parameters

Hyperparameter	Symbol	Value	Rationale
Common space dimension	d_c	64	JL lemma: $\varepsilon \approx 0.3$ at prob ≈ 0.73 ; balances projection speed and reliability
Outer iterations (BCD)	n_{outer}	5	Sufficient for convergence (Duan et al., 2012, Table 2: 3–10 iterations typically sufficient)
Subgradient learning rate	η	10^{-3}	Small enough for projection gradient not to overshoot the Frobenius boundary
Squared-hinge coefficient	C	1.0	Standard value for LinearSVC

Hyperparameter	Symbol	Value	Rationale
Soft Frobenius regularizer	λ_P, λ_Q	10^{-3}	Light regularization, allowing the hard constraint to serve as the primary control
Hard Frobenius radius	r_P, r_Q	10.0	Bounds the magnitude of variation; large enough to avoid being overly restrictive
SVM dual	-	False	$n_{\text{features}} = 324\text{K} \gg n_{\text{samples}} = 187$; primal is faster
SVM class_weight	-	"balanced"	Corrects class imbalance (LC ~21.4%, TDCCC ~22.1% default)
SVM max_iter	-	2000	Ensures internal SVM convergence
Platt calibration $\mathcal{C}$	-	10^6	Near-zero regularization $\approx$ pure MLE
Source sample size	-	300000	Fixed; stratified subsample from 1,325,535 records

4.3. Domain-Adversarial Neural Network (DANN) Methodology

4.3.1. H-divergence theory

The theoretical foundation of DANN rests on the target risk bound of Ben-David et al. (2010, Theorem 2):

$$\varepsilon_T(h) \leq \varepsilon_S(h) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(\mathcal{D}_S, \mathcal{D}_T) + \lambda^*$$

where:

- $\varepsilon_T(h)$: classification risk on the target domain - the quantity to be minimized.
- $\varepsilon_S(h)$: classification risk on the source domain - minimized via supervised training on source labels.
- $d_{\mathcal{H}\Delta\mathcal{H}}(\mathcal{D}_S, \mathcal{D}_T)$: $\mathcal{H}$ -divergence - measures the discrepancy between the two domain distributions in the representation space.
- λ^* : combined error of the ideal joint hypothesis - typically small and not directly controllable.

To define $\mathcal{H}$ -divergence formally, let $\mathcal{H}\Delta\mathcal{H}$ denote the hypothesis symmetric difference class:

$$\mathcal{H}\Delta\mathcal{H} = \{h(x) \oplus h'(x) : h, h' \in \mathcal{H}\}$$

$\mathcal{H}\Delta\mathcal{H}$ is the set of all hypotheses capable of "detecting" discrepancies between $\mathcal{D}_S$ and $\mathcal{D}_T$. The $\mathcal{H}$ -divergence is then defined as:

$$d_{\mathcal{H}\Delta\mathcal{H}}(\mathcal{D}_S, \mathcal{D}_T) = 2 \sup_{h \in \mathcal{H}} | \Pr_{x \sim \mathcal{D}_S} [h(x) = 1] - \Pr_{x \sim \mathcal{D}_T} [h(x) = 1] |$$

Computationally, $d_{\mathcal{H}\Delta\mathcal{H}}$ cannot be evaluated directly from data. Ganin (2016, §3) shows that it can be approximated by training a binary classifier to distinguish source samples (label 0) from target samples (label 1): if the domain classifier achieves accuracy $\hat{\varepsilon}$, the proxy estimate is $\hat{d} = 2(1 - 2\hat{\varepsilon})$. DANN minimizes $\hat{d}$ by preventing the domain classifier from distinguishing source from target - driving $\hat{\varepsilon}$ toward 0.5 (random guess), which corresponds to $\hat{d} \rightarrow 0$.

The bound identifies two levers for reducing target risk: (i) reducing $\varepsilon_S(h)$ through supervised learning, and (ii) reducing $\mathcal{H}$ -divergence - rendering the representations of the two domains indistinguishable (domain-invariant). DANN simultaneously pursues both objectives.

The bound is informative only when λ^* is small - that is, when a hypothesis exists that performs well on both domains. If the two domains are structurally too dissimilar (e.g., if the default dynamics of the US P2P lending market differ fundamentally from those of Taiwanese credit cards), λ^* grows large and the bound becomes vacuous - the theoretical root cause of negative transfer.

4.3.2. DANN Principle (Ganin et al., 2016)

Domain-Adversarial Neural Network (DANN) is a neural network comprising three components - feature extractor, label predictor, and domain classifier - trained jointly under a min-max objective that drives the encoder to produce domain-invariant features. Concretely, DANN operationalizes the bound above through the following problem:

$$\min_{G_f, G_y} \max_{G_d} \mathcal{L}_y(G_y(G_f(x_s)), y_s) - \lambda \cdot \mathcal{L}_d(G_d(G_f(x)), d)$$

where:

- G_f : feature extractor (encoder) - maps inputs into the representation space $\mathbb{R}^{128}$.
- G_y : label predictor - predicts the default/non-default label from the representation.
- G_d : domain classifier - distinguishes source vs. target from the representation.
- d : domain label (source = 0, target = 1) - not the prediction target; domain labels indicate only which dataset a sample comes from, not whether it defaults.
- λ : balancing coefficient between classification loss and domain adversarial loss, varied according to a warm-up schedule.

This min-max formulation defines a two-player game: G_f and G_y jointly minimize $\mathcal{L}_y$ - the encoder and classifier learn to predict default accurately on the source; G_d maximizes $\mathcal{L}_d$ - the domain classifier learns to distinguish source from target; but G_f simultaneously maximizes $\mathcal{L}_d$ (through the GRL) - the encoder attempts to fool the domain classifier. At the Nash equilibrium, the encoder produces representations that the domain classifier cannot distinguish as source or target, reducing $\mathcal{H}$ -divergence and tightening the bound.

Ganin et al. (2016) designed DANN under the unsupervised DA principle: only source labels y_s are used for the classification loss. Target labels y_t are not used during training - they are reserved exclusively for post-training evaluation. This reflects the practical scenario where

labeled data are available from the old market (source) and the goal is to extend to a new market (target) where labeled data are scarce.

4.3.3. Gradient Reversal Layer (GRL)

The min-max problem above ordinarily requires two-step optimization - alternately updating the generator (encoder + classifier) and the discriminator (domain classifier), analogously to a GAN (Goodfellow et al., 2014). The GRL (Ganin & Lempitsky, 2015) solves it in a single backward pass:

$$\text{GRL: forward: } \mathbf{x} \mapsto \mathbf{x}, \text{backward: } \frac{\partial \mathcal{L}}{\partial \mathbf{x}} \mapsto -\lambda \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{x}}$$

As gradients flow backward through the GRL:

- Domain classifier G_d : receives unmodified gradients $\rightarrow$ learns to distinguish source from target (MIN domain loss).
- Feature extractor G_f : receives sign-reversed gradients through the GRL $\rightarrow$ learns to fool the domain classifier (MAX domain loss = reduce $\mathcal{H}$ -divergence).
- Label predictor G_y : does not pass through the GRL $\rightarrow$ receives unmodified gradients for the classification loss.

The negative sign before $\lambda \mathcal{L}_d$ in the theoretical formulation and the positive sign in `train_dann()` (written as $L = L_s + L_d$) are not contradictory. In the forward pass, $\mathcal{L}_d$ is an ordinary loss - the domain classifier seeks to minimize it to distinguish source from target, and the GRL does not intervene in the forward pass (`GRL.forward = identity`). In the backward pass, the gradient of $\mathcal{L}_d$ is negated upon passing through the GRL - the encoder receives a reversed gradient, effectively "seeing" itself as maximizing $\mathcal{L}_d$. Consequently, the negative sign in the theoretical objective reflects the encoder's perspective; in the code, the optimizer sees $L_s + L_d$ with a positive sign because the GRL handles gradient reversal automatically. The two competing objectives are thus resolved in a single forward/backward pass without alternating loops.

The GRL offers three advantages over two-step optimization: (i) a single optimizer suffices, (ii) all parameters are updated simultaneously, and (iii) it is straightforward to implement in PyTorch.

4.3.4. HeteroDANN Architecture

Since the two domains differ in dimensionality, the original DANN architecture with a shared encoder cannot be applied directly. HeteroDANN uses two domain-specific encoders, each processing the input of its corresponding domain and thesising it into a shared representation space $\mathbb{R}^{128}$:

Source input ($d_s = 120$) $\rightarrow$ Encoder_S $\rightarrow \mathbf{h} \in \mathbb{R}^{128} \neg$

or ($d_s = 37$) $\quad \quad \quad \vdash \rightarrow$ Label Predictor $\rightarrow$ logit (default Y/N)

Target input ($d_t = 23$) $\rightarrow$ Encoder_T $\rightarrow h \in \mathbb{R}^{128}$ $\perp$

$\perp \rightarrow$ GRL $\rightarrow$ Domain Classifier $\rightarrow$ logit (source/target)

Table 9: Four modules of DANN

Module	Input $\rightarrow$ Output	Architecture
DomainEncoder_S	$\mathbb{R}^{d_s} \rightarrow \mathbb{R}^{128}$	Linear $\rightarrow$ LN $\rightarrow$ ReLU $\rightarrow$ Dropout(0.2) $\rightarrow$ Linear $\rightarrow$ LN $\rightarrow$ ReLU
DomainEncoder_T	$\mathbb{R}^{d_t} \rightarrow \mathbb{R}^{128}$	Linear $\rightarrow$ LN $\rightarrow$ ReLU $\rightarrow$ Dropout(0.2) $\rightarrow$ Linear $\rightarrow$ LN $\rightarrow$ ReLU
LabelPredictor	$\mathbb{R}^{128} \rightarrow \mathbb{R}^1$	Linear(128,32) $\rightarrow$ ReLU $\rightarrow$ Linear(32,1)
DomainClassifier	$\mathbb{R}^{128} \rightarrow \mathbb{R}^1$ (qua GRL)	GRL $\rightarrow$ Linear(128,32) $\rightarrow$ ReLU $\rightarrow$ Linear(32,1)

Two key architectural decisions govern the encoder design.

First, LayerNorm instead of BatchNorm (Ba, Kiros & Hinton, 2016): each mini-batch in DANN contains both source and target samples. BatchNorm computes mean and variance across the entire batch, causing statistics from the two domains to be mixed - blurring the distribution boundary and hindering the encoder's ability to learn domain-invariant representations. LayerNorm normalizes along the feature dimension of each individual sample, independently of the batch composition - making it appropriate for mixed-domain batches.

Second, Kaiming initialization (He et al., 2015): $\sigma^2 = 2/n_{in}$ for ReLU activations. Glorot/Xavier initialization uses $\sigma^2 = 1/n_{in}$, but since ReLU zeroes out half of its inputs, the effective variance is halved at each layer. Kaiming compensates with a factor of 2, ensuring variance remains stable across deep layers.

4.3.5. Training Schedule (Ganin et al., 2016, §4.2)

Lambda schedule (adversarial warm-up):

$$\lambda(p) = \frac{2}{1 + \exp(-\gamma p)} - 1, p = \frac{\text{step}}{\text{total_steps}}, \gamma = 10$$

This curve guarantees that $\lambda(0) = 0$ - no adversarial signal at the start, allowing the encoder to first learn a basic representation from source classification; $\lambda(0.5) \approx 0.987$ - rapid increase (sigmoid shape); $\lambda(1) \approx 1$ - saturation. The warm-up is a critical design choice: if the adversarial signal is applied at full strength from the outset ($\lambda = 1$), the encoder has not yet formed a useful representation, making it easy for the domain classifier to distinguish domains - large reversed gradients then destabilize training. The warm-up schedule also serves as a preventive strategy against negative transfer.

Learning rate schedule (polynomial decay):

$$\eta(p) = \frac{\eta_0}{(1 + \alpha p)^\beta}, \eta_0 = 0.01, \alpha = 10, \beta = 0.75$$

Polynomial decay is slower than exponential decay but faster than $1/t$, making it well-suited to adversarial training where an excessively large learning rate late in training can induce oscillation.

4.3.6. Combined Loss Function and Training Loop

The total loss at each step (Ganin, 2016, Eq. 1–4):

$$\mathcal{L} = \mathcal{L}_s(\hat{y}_s, y_s; w_s) + \lambda \cdot \mathcal{L}_d(\hat{d}, d)$$

where:

- $\mathcal{L}_s$: BCEWithLogitsLoss for classification loss, computed on source samples only. $\text{pos_weight} = N_{\text{neg}}/N_{\text{pos}} \approx 6.14$ (derived automatically from the LC default rate of approximately 21.39%) to correct for class imbalance.
- $\mathcal{L}_d$: BCEWithLogitsLoss for domain loss, computed on both source and target. Source samples receive domain label = 0; target samples receive domain label = 1.

The GRL sits between the encoder output and the domain classifier input. Although the theoretical formulation above writes $L = \mathcal{L}_s + \lambda \cdot \mathcal{L}_d$, the code computes the forward loss as $L = \mathcal{L}_s + \mathcal{L}_d$ without an explicit λ coefficient - because the GRL is an identity in the forward pass. The scaling by λ occurs entirely in the backward pass: the GRL sign-reverses and scales by λ all gradients from $\mathcal{L}_d$ propagating down to the encoder, so the encoder effectively optimises $\mathcal{L}_s - \lambda \cdot \mathcal{L}_d$. This single-pass mechanism is mathematically equivalent to the min-max formulation; the $\lambda \cdot \mathcal{L}_d$ term in the formula above therefore represents the encoder’s effective gradient contribution, not a literal multiplication in the forward computation.

The `train_dann()` function implements step-based training (not epoch-based, following Ganin, 2016). At each step from 1 to `total_steps`: compute $\lambda(p)$ and $\eta(p)$; update the learning rate in the optimizer; sample a source batch and a target batch; forward pass through both encoders; compute label predictions and domain predictions via the GRL; compute the loss; backward pass with gradient clipping `max_norm = 1.0` (Pascanu, Mikolov & Bengio, 2013) to guard against exploding gradients when the GRL reverses large gradients; and optimizer step.

Experimental parameters: `total_steps = 4000`, `batch_size = 64`, `hidden_dim = 128`, optimizer Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, Kingma & Ba, 2015).

Notably, there is no $\mathcal{L}_t$ (target classification loss) in the total loss - target labels y_t do not participate in training. This is a fundamental distinction from HFA (Chapter 5), where both y_s and y_t are used in the optimization objective.

4.3.7. Prediction and Evaluation

At evaluation time, `predict_dann()` runs only `DomainEncoder_T` together with `LabelPredictor` - the GRL and `DomainClassifier` are not involved. The output logit is passed through a sigmoid to obtain the probability $\hat{p}(y = 1 \mid \mathbf{x}_T)$; the KS-optimal threshold is determined on the training fold and applied to the test fold for computing AUC, KS, Brier Score, Sensitivity, and Specificity

4.3.8. DANN experimental design

Table 10: DANN method's parameters

Hyperparameter	Value	Rationale
hidden_dim	128	Sufficiently expressive for 120/23 input features; not large enough to cause overfitting
dropout	0.2	Light regularization - randomly deactivates 20% of neurons per forward pass
total_steps	4000	Step-based training (not epoch-based); with batch_size=64, approximately 260 "epochs" over the target set
batch_size	64	Balances gradient noise (small $\rightarrow$ noisy $\rightarrow$ implicit regularization) and computational speed
initial_lr (η_0)	0.01	Following Ganin (2016, §4.2)
γ (lambda schedule)	10	Following Ganin (2016, §4.2); controls the adversarial warm-up rate
α (LR decay)	10	Following Ganin (2016, §4.2)
β (LR decay)	0.75	Following Ganin (2016, §4.2)
optimizer	Adam	$\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ (Kingma & Ba, 2015); Adam is stable for adversarial training due to per-parameter adaptive learning rates
max_grad_norm	1.0	Gradient clipping (Pascanu et al., 2013); the GRL can reverse large gradients - clipping limits the risk of exploding gradients
Initialization	Kaiming	fan_in mode, ReLU nonlinearity (He et al., 2015)
Normalization	LayerNorm	Ba et al. (2016); batch-independent, suitable for mixed-domain batches
Source sample size	300000	Fixed; consistent with HFA

4.4. Negative Transfer Ablation Design

Each transfer learning method is evaluated across three cases on the hold-out test set (6000 samples), with the aim of isolating the contribution of each component:

Case 1 - Target only (no transfer): Training is performed exclusively on TDCCC, with no use of Lending Club data. This serves as the method-specific baseline (HFA uses LinearSVC; DANN uses Encoder_T + LabelPredictor). If either Case 2 or Case 3 underperforms Case 1, negative transfer is confirmed.

Case 2 - Naive concatenation (naive transfer): Lending Club and TDCCC are combined without any alignment mechanism - block-diagonal concatenation for HFA, and zero-padding the source to the target's dimensionality for DANN. This tests whether simply adding source data, absent any alignment, is beneficial.

Case 3 - Full method: The complete transfer learning method is applied - HFA with Block Coordinate Descent and Frobenius projection, or DANN with the GRL and adversarial training. This tests whether the alignment mechanism provides additional gains over naive transfer.

The three-case protocol enables clear attribution: Case 3 vs. Case 1 measures the net benefit of the full transfer method; Case 3 vs. Case 2 isolates the contribution of the alignment mechanism specifically; and Case 2 vs. Case 1 measures the effect of source data alone. If Case 3 fails to outperform Case 1, the source–target pair is deemed unsuitable for transfer under the given method.

Chapter 5. Result, Discussion and Implications

5.1. Baseline results

Three baseline models - Logistic Regression (LR), Random Forest (RF), and XGBoost - were trained on the 24000-sample TDCDC train pool and evaluated on the 6000-sample hold-out test set, following the benchmark protocol of Lessmann et al. (2015).

5.1.1. Cross-validation results

5-fold stratified cross-validation results on the train pool show that all three models achieve stable and consistent performance across folds.

Table 11: Baseline method - Cross-validation (5-fold) results

Model	AUC	KS	Brier	Sensitivity	Specificity	Accuracy	Threshold
Logistic Regression	0.7262	0.3737	0.2058	0.5413	0.8323	0.7680	0.547
Random Forest	0.7790	0.4281	0.1756	0.6387	0.7894	0.7561	0.457
XGBoost	0.7819	0.4319	0.1771	0.6257	0.8062	0.7662	0.502

5.1.2. Hold-out results

Table 12: Baseline method - Hold-out results

Hold-out evaluation (6000 samples) confirms the trends observed in cross-validation.

Model	AUC	KS	Brier	Sensitivity	Specificity	Accuracy	Threshold
Logistic Regression	0.7084	0.3586	0.2089	0.5207	0.8376	0.7675	0.552
Random Forest	0.7739	0.4333	0.1786	0.6420	0.7734	0.7443	0.449
XGBoost	0.7758	0.4235	0.1801	0.6586	0.7541	0.7330	0.461

Model	TP/TN/FP/FN
Logistic Regression	691/3914/759/636
Random Forest	852/3614/1059/475
XGBoost	874/3524/1149/453

5.1.3. Baseline Analysis

XGBoost achieves the highest AUC on the hold-out set (0.7758), followed by Random Forest (0.7739) and Logistic Regression (0.7084). The gap between cross-validation and hold-out AUC is negligible - XGBoost drops by only 0.0061 (from 0.7819 to 0.7758), and Random Forest by 0.0051 - indicating no substantial overfitting. These results are consistent with prior benchmarks (Lessmann et al., 2015) and align closely with the $AUC \approx 0.77$ reported by Yeh and Lien (2009) on the same dataset.

Notably, Random Forest leads on KS (0.4333) - the metric most widely used in banking practice (Thomas et al., 2002) - while XGBoost leads on AUC (0.7758) and Sensitivity (0.6586). This divergence reflects the algorithmic properties of the two methods: Random Forest optimizes variance reduction through bagging, whereas XGBoost optimizes bias reduction through sequential boosting (Friedman, 2001). XGBoost is selected as the representative baseline ($AUC = 0.7758$) for comparison with HFA and DANN in subsequent sections - any transfer learning method falling below this threshold is treated as evidence of negative transfer (Pan & Yang, 2010).

5.2. HFA results

Heterogeneous Feature Augmentation (HFA; Yang et al., 2020, Eq. 6.8) uses 300000 Lending Club samples as the source domain and 24000 TDCCC samples as the target domain. Unlike DANN, HFA is a supervised domain adaptation method - labels from both domains (y_s and y_t) are used during training.

Case 1 (Full LC)

Table 13: HFA method - Full LC results

Fold	AUC	KS	Brier	Sensitivity	Specificity	Accuracy	Threshold
1	0.7287	0.3906	0.2061	0.5287	0.8561	0.7837	0.568
2	0.7205	0.3895	0.2042	0.5659	0.8058	0.7527	0.530
3	0.7078	0.3589	0.2101	0.5395	0.8133	0.7527	0.538
4	0.7088	0.3644	0.2090	0.5009	0.8585	0.7794	0.576
5	0.7213	0.3754	0.2058	0.4953	0.8777	0.7931	0.581
Mean	0.7174 ± 0.0089	0.3758 ± 0.0143	0.2070 ± 0.0024	0.5261 ± 0.0290	0.8423 ± 0.0311	0.7723 ± 0.0186	0.5586 ± 0.0231

Case 2 (Selected LC)

Table 14: HFA method - Selected LC results

Fold	AUC	KS	Brier	Sensitivity	Specificity	Accuracy	Threshold
1	0.7370	0.3970	0.2047	0.5485	0.8411	0.7765	0.551
2	0.7232	0.3940	0.2038	0.5546	0.8242	0.7646	0.542
3	0.7092	0.3617	0.2097	0.5424	0.8087	0.7498	0.538
4	0.7134	0.3734	0.2079	0.5706	0.7975	0.7473	0.529
5	0.7264	0.3811	0.2049	0.5028	0.8646	0.7846	0.567
Mean	0.7218 ± 0.0110	0.3814 ± 0.0146	0.2062 ± 0.0025	0.5438 ± 0.0252	0.8272 ± 0.0266	0.7646 ± 0.0163	0.5454 ± 0.0144

The AUC difference is only +0.0044 and the KS difference +0.0056 - both within the fold-to-fold variance range (std $\approx$ 0.009–0.011), and therefore not statistically significant.

Case 1 detects defaults more effectively (Sensitivity +0.0177), but at the cost of a higher false positive rate - Specificity is lower by 0.0151. Case 2 exhibits the opposite profile: more conservative, with fewer false alarms - better suited to credit scoring contexts where the cost of false negatives is high but false positives must also be controlled.

Reducing the feature space by approximately 69% (from 120 to 37 features) does not meaningfully degrade discriminative performance, supporting the argument of Pan and Yang (2010) that a more compact feature space helps reduce cross-domain noise. Case 2 carries the additional advantages of greater interpretability and lower feature acquisition costs in real-world deployment.

5.3. DANN Results

Domain-Adversarial Neural Network (DANN; Ganin et al., 2016) is implemented as HeteroDANN with two domain-specific encoders handling $\mathcal{X}_S$ (Lending Club, $d_S \in \{120, 37\}$) and $\mathcal{X}_T$ (TDCCC, $d_T = 23$) respectively. This is an unsupervised domain adaptation method - only source labels y_s are used during training; target labels y_t do not participate in the optimization.

Case 1 (Full LC)

Table 15: DANN method - Full LC results

Fold	AUC	KS	Brier	Sensitivity	Specificity	Accuracy	Threshold
1	0.4222	0.0083	0.3363	0.9689	0.0348	0.2412	0.586

Fold	AUC	KS	Brier	Sensitivity	Specificity	Accuracy	Threshold
2	0.4707	0.0559	0.2473	0.3785	0.6747	0.6092	0.476
3	0.6009	0.2007	0.2510	0.5659	0.6281	0.6144	0.533
4	0.6454	0.2769	0.4710	0.4680	0.8066	0.7317	0.920
5	0.4165	0.0408	0.2783	0.9388	0.0928	0.2800	0.489
Mean	0.5111 ± 0.0944	0.1165 ± 0.1039	0.3168 ± 0.0933	0.6640 ± 0.2730	0.4474 ± 0.3568	0.4953 ± 0.2202	0.601 ± 0.184

Case 2 (Selected LC)

Table 16: DANN method - Selected LC results

Fold	AUC	KS	Brier	Sensitivity	Specificity	Accuracy	Threshold
1	0.4222	0.0083	0.3363	0.9689	0.0348	0.2412	0.586
2	0.4707	0.0559	0.2473	0.3785	0.6747	0.6092	0.476
3	0.6009	0.2007	0.2510	0.5659	0.6281	0.6144	0.533
4	0.6454	0.2769	0.4710	0.4680	0.8066	0.7317	0.920
5	0.4165	0.0408	0.2783	0.9388	0.0928	0.2800	0.489
Mean	0.5111 ± 0.0944	0.1165 ± 0.1039	0.3168 ± 0.0933	0.6640 ± 0.2730	0.4474 ± 0.3568	0.4953 ± 0.2202	0.601 ± 0.184

The mean AUC reaches only approximately 0.50–0.51 - equivalent to a random classifier (AUC = 0.5) - and KS < 0.12 offers no practical discriminative value in credit scoring, where the accepted minimum is typically KS ≥ 0.20.

The AUC standard deviation reaches ±0.10–0.12, compared to ±0.008–0.009 for HFA. Most critically, Fold 3 Case 1 exhibits Sensitivity = 0.000 and Specificity ≈ 1.000 - a complete model collapse in which all samples are predicted as non-default.

DANN was designed for domain adaptation under the homogeneous setting, where source and target share the same feature space - a condition that does not hold in the LC → TDCCC problem, which is inherently a heterogeneous transfer scenario with entirely distinct feature

spaces across the two domains. The Gradient Reversal Layer cannot align two distributions whose input representations are structurally incompatible, leading to unstable adversarial training and severe negative transfer (Ganin et al., 2016).

5.4. Ablation study results

The ablation study is designed to isolate the contribution of each transfer learning component. Three cases are defined consistently for both HFA and DANN:

- Case 1 - Target only (no transfer): trained exclusively on TDCCC, with no Lending Club data.
- Case 2 - Naive concatenation (naive transfer): Lending Club and TDCCC are combined without any alignment mechanism.
- Case 3 - Full method (full transfer): HFA with Block Coordinate Descent, or DANN with GRL and adversarial training.

5.4.1. HFA ablation results

Case 1 (Full LC)

Table 17: HFA ablation - Full LC results

Case	AUC	KS	Brier	Sensitivity	Specificity	Accuracy	Threshold
Case 1 - Target only	0.7076	0.3572	0.2089	0.5162	0.8395	0.7680	0.554
Case 2 - Concat naive	0.7077	0.3571	0.2128	0.6405	0.6653	0.6598	0.498
Case 3 - Full HFA	0.6993	0.3613	0.2100	0.5011	0.8571	0.7783	0.570

Case 2 (Selected LC)

Table 18: HFA ablation - Selected LC results

Case	AUC	KS	Brier	Sensitivity	Specificity	Accuracy	Threshold
Case 1 - Target only	0.7076	0.3572	0.2089	0.5162	0.8395	0.7680	0.554
Case 2 - Concat naive	0.7077	0.3576	0.2132	0.6458	0.6574	0.6548	0.496
Case 3 - Full HFA	0.7042	0.3595	0.2092	0.5290	0.8226	0.7577	0.541

Across both Case 1 and Case 2, the three HFA cases yield nearly identical AUC (~ 0.70 – 0.71), indicating that BCD alignment does not generate additional discriminative signal beyond the target-only baseline. However, Full HFA (Case 3) produces marginal KS improvements ($+0.0041$ over Case 1, $+0.0023$ over Case 2) and higher Specificity relative to naive concatenation, suggesting that feature alignment has a positive effect on threshold calibration and false positive control - even if insufficient to surpass the absolute baseline. This is consistent with theory: under large domain gap, transfer can at best maintain - rather than necessarily improve - performance, a condition known as safe transfer (Pan & Yang, 2010).

5.4.2. DANN ablation results

Case 1 (Full LC)

Table 19: DANN ablation - Full LC results

Case	AUC	KS	Brier	Sensitivity	Specificity	Accuracy	Threshold
Case 1 - Target only	0.7659	0.4013	0.1841	0.6345	0.7625	0.7342	0.449
Case 2 - Concat naive	0.6837	0.3080	0.2393	0.4431	0.8615	0.7690	0.612
Case 3 - Full DANN	0.4655	0.0530	0.1949	0.7476	0.3019	0.4005	0.124

Case 2 (Selected LC)

Table 20: DANN ablation - Selected LC results

Case	AUC	KS	Brier	Sensitivity	Specificity	Accuracy	Threshold
Case 1 - Target only	0.7681	0.4012	0.1884	0.6451	0.7526	0.7288	0.458
Case 2 - Concat naive	0.6814	0.3080	0.2314	0.4476	0.8575	0.7668	0.600
Case 3 - Full DANN	0.6167	0.2095	0.3281	0.4928	0.7143	0.6653	0.649

The DANN ablation results show that the Case 1 model (target only $\lambda = 0$,) achieves AUC ~ 0.77 - the highest across all configurations - while Full DANN (Case 3) collapses to AUC = 0.4655 on Case 1 and 0.6167 on Case 2. Inspection of training logs reveals $\mathcal{L}_d \approx 0.000$ from approximately step 200 onwards. Because $\mathcal{L}_d$ is a binary cross-entropy with logits, $\mathcal{L}_d \rightarrow 0$ indicates that the domain classifier separates source from target with very high confidence - the opposite of failing to learn. The failure lies on the encoder side: the encoder cannot produce features confusing enough to fool a domain classifier that observes inputs of radically different structure ($d_S = 120$ versus $d_T = 23$ channels feeding two separate encoders). Once $\sigma(\text{logit}_d)$

saturates near 1, the gradient $\frac{\partial \mathcal{L}_d}{\partial \text{logit}_d}$ becomes vanishingly small; the Gradient Reversal Layer has nothing meaningful to reverse, and the encoder receives no useful adversarial signal. The label loss $\mathcal{L}_s$ is therefore optimised on the source distribution alone, and the encoder learns features that do not generalise to the target - yielding the observed Case 3 collapse. This is an instance of gradient saturation under heterogeneous adversarial alignment, not feature collapse in the classical GAN sense, and is what makes DANN optimisation-hard (rather than mathematically ill-posed) when d_S and d_T differ substantially.

5.4.3. Cross-method ablation comparison

Table 21: Ablation comparison

Method	Feature Set	Case 1 AUC	Case 2 AUC	Case 3 AUC	Trend C1→C3
HFA	Case 1	0.7076	0.7077	0.6993	↓ small (−0.0083)
HFA	Case 2	0.7076	0.7077	0.7042	↓ very small (−0.0034)
DANN	Case 1	0.7659	0.6837	0.4655	↓↓ very big (−0.3004)
DANN	Case 2	0.7681	0.6814	0.6167	↓ big (−0.1514)

The ablation study confirms two contrasting findings: HFA achieves safe transfer with negligible degradation ($\Delta\text{AUC} < 0.009$), while DANN induces severe negative transfer on Case 1 ($\Delta\text{AUC} = -0.30$) and moderate negative transfer on Case 2 ($\Delta\text{AUC} = -0.15$). The gap between DANN[Case 1] and DANN[Case 2] in Case 3 (AUC: 0.4655 vs. 0.6167) provides direct empirical evidence that semantically guided feature reduction can partially mitigate negative transfer even under an adversarial architecture that is fundamentally ill-suited to the heterogeneous domain setting - consistent with the recommendation of Pan and Yang (2010, §3.3).

5.5. Summary of key findings

5.5.1. Overall comparison

Table 22: Overall comparison (1)

Method	Feature set	AUC	KS	Brier	Sensitivity	Specificity	Accuracy
Logistic Regression	Case 1 / Case 2	0.7084	0.3586	0.2089	0.5207	0.8376	0.7675
Random Forest	Case 1 / Case 2	0.7739	0.4333	0.1786	0.6420	0.7734	0.7443
XGBoost (<i>best baseline</i>)	Case 1 / Case 2	0.7758	0.4235	0.1801	0.6586	0.7541	0.7330

Method	Feature set	AUC	KS	Brier	Sensitivity	Specificity	Accuracy
HFA (Full)	Case 1	0.6993	0.3613	0.2100	0.5011	0.8571	0.7783
HFA (Full)	Case 2	0.7042	0.3595	0.2092	0.5290	0.8226	0.7577
DANN (Full)	Case 1	0.4655	0.0530	0.1949	0.7476	0.3019	0.4005
DANN (Full)	Case 2	0.6167	0.2095	0.3281	0.4928	0.7143	0.6653

Across both feature sets, XGBoost achieves $\text{AUC} = 0.7758$ - the highest overall and the reference baseline. HFA falls below this baseline by -7.65 pp (Case 1) and -7.16 pp (Case 2); DANN by -31.03 pp (Case 1) and -15.91 pp (Case 2). This constitutes empirical evidence of negative transfer in both methods, arising from the large domain gap between Lending Club (P2P lending, United States) and TDCCC (traditional bank, Taiwan) in the heterogeneous transfer learning setting.

HFA maintains performance close to the baseline with minimal AUC degradation (~ 7 pp), KS still reaching 0.36, and Specificity exceeding XGBoost (0.8571 vs. 0.7541 on Case 1) - indicating that BCD alignment provides better false positive control. DANN on Case 1 collapses to $\text{AUC} = 0.4655$, equivalent to a random classifier. The training log shows $\mathcal{L}_d \approx 0$ throughout training, indicating that the domain classifier separates source from target near-perfectly; the encoder cannot fool it, gradients through the Gradient Reversal Layer saturate, and no useful domain-alignment signal is propagated back.

Table 23: Overall comparison (2)

Method	ΔAUC (Case 2 – Case 1)	Mechanism
HFA	+0.49 pp	Removing noisy features reduces BCD projection error
DANN	+15.12 pp	Semantic feature alignment narrows the domain gap, allowing $\mathcal{L}_d$ to escape collapse

The performance improvement from Case 1 to Case 2 is substantially larger for DANN (+15.12 pp) than for HFA (+0.49 pp), confirming that semantically guided feature reduction plays a particularly critical role when the transfer architecture relies on a shared representation space (DANN) - an important design implication for future heterogeneous transfer learning systems.

Table 24: Overall comparison (3)

Method	Feature Set	AUC (CV)	Std	KS (CV)	Std
HFA	Case 1	0.7174	± 0.0080	0.3758	± 0.0128
HFA	Case 2	0.7218	± 0.0098	0.3814	± 0.0131

Method	Feature Set	AUC (CV)	Std	KS (CV)	Std
DANN	Case 1	0.5048	± 0.1103	0.1053	± 0.1087
DANN	Case 2	0.5111	± 0.0944	0.1165	± 0.1039

The standard deviation of DANN is approximately 12–14 times larger than that of HFA (± 0.10 vs. ± 0.009), reflecting structural instability rather than random noise. HFA, by contrast, exhibits consistently low variance across folds - a critical property in production credit scoring, where model reliability over time and across data segments is considered of equal importance to absolute AUC (Siddiqi, 2006).

5.5.2. Root cause analysis of Negative transfer

Finally, the negative transfer in both methods may be attributed to three root causes closely tied to the theoretical framework of Ben-David et al. (2010):

Firstly, there is an excessive domain gap (λ^* too large). Lending Club (P2P lending, US market, 2007–2018) and TDCCC (credit cards, Taiwan market, 2005) have fundamentally different default dynamics: LC reflects voluntary consumer borrowing behavior characterized by FICO scores and debt-to-income ratios, while TDCCC captures credit card repayment behavior through detailed PAY_X payment histories. Compounding this structural heterogeneity, the two datasets are temporally inverted relative to the conventional transfer learning assumption. In standard practice, knowledge flows from a richer, historically established source to a newer target domain; here, however, the target domain (TDCCC, 2005) predates the source domain (LC, 2007–2018) by at least two years, meaning the source data cannot encode any credit-cycle dynamics that the target has already experienced. Differences in credit product characteristics lead to distributional shifts that substantially degrade model accuracy. When λ^* (the combined error of the ideal joint hypothesis) is large, the Ben-David bound becomes vacuous: no single hypothesis can perform well on both domains simultaneously. The dimensionality ratio d_S/d_T further reflects the structural asymmetry between the two feature spaces.

Secondly, HFA uses a LinearSVC in the augmented feature space (as in Case 1). Linear methods are known to be inadequate when the distributional difference between domains is itself nonlinear; as Pan et al. (2011) show, effective domain adaptation requires finding a latent space in which both marginal and conditional distributions are jointly reduced, something a simple linear projection cannot guarantee. When the two domain distributions diverge as substantially as they do here, the shared block ϕ fails to converge to any meaningful common representation. The Frobenius norm constraint $\|\Delta\|_F \leq \epsilon$ may additionally be too restrictive for ϕ , limiting the projection amplitude and preventing the model from exploring transformations that could bridge the gap.

Thirdly, the training log shows $\mathcal{L}_d \approx 0$ from step 200 onwards, which means the domain classifier identifies source and target with near-perfect confidence - not the opposite. Two encoders with different input dimensionalities (120 versus 23) produce embeddings that, despite the encoder's compression to $\mathbb{R}^{128}$, retain enough dimension-specific signal for the domain classifier to separate trivially. Once $\sigma(\text{logit}_d) \approx 1$, the gradient through the Gradient Reversal

Layer collapses toward zero, and the encoder receives essentially no adversarial signal. The label loss $\mathcal{L}_s$ is then optimised on source data alone, the resulting representation does not generalise to the target, and Case 3 performance falls to near-random (AUC = 0.4655 on Case 1; partially recovered to 0.6167 on Case 2 because reduced d_s narrows the easy-separation gap).

Chapter 6. Conclusion

6.1. Research summary

This thesis systematically implemented and evaluated two representative Heterogeneous Domain Adaptation methods - HFA (Yang et al., 2020, Eq. 6.8; Duan, Xu & Tsang, 2012) and DANN (Ganin et al., 2016) - for the credit scoring problem on the Lending Club (source) and Default of Credit Card Clients - TDCCC (target) dataset pair. The experimental pipeline comprised: (i) rigorous preprocessing under two feature set strategies (Case 1: 121 full features; Case 2: 37 features selected according to the 5 C's of Credit); (ii) three baselines (LR / RF / XGBoost) as reference benchmarks; (iii) training and evaluation of both transfer methods via 5-fold cross-validation and hold-out; and (iv) a 3-case ablation to explicitly test for negative transfer.

6.2. HFA method evaluation

HFA trains stably: BCD converges within 5 outer loop iterations, Frobenius projection operates correctly (consistent across all folds and the hold-out set), and Platt calibration yields valid probability outputs. Predictive performance is acceptable but does not surpass the baseline: hold-out AUC reaches 0.7042 (Case 2) / 0.6993 (Case 1) - lower than the strongest baseline (XGBoost: 0.7758) by -7.16 pp and -7.65 pp respectively, and below Case 1 (LinearSVC on TDCCC alone: 0.7076). 5-fold CV yields stable results with $\text{AUC} = 0.7218 \pm 0.0098$ (Case 2) and 0.7174 ± 0.0080 (Case 1) - very low variance, confirming the method's consistency.

The probable causes of negative transfer are: the linear constraint on P, Q is too restrictive for the wide domain gap between LC and TDCCC; and LinearSVC in the augmented space of dimension $k + d_S + d_T$ may be statistically sparse given only ~ 300000 source samples and ~ 24000 target samples within a heterogeneous feature space.

Advantages of HFA:

- Convex-per-block framework with clear convergence guarantees (Tseng, 2001, Theorem 4.1).
- Low computational cost, runnable on CPU.
- Low variance (~ 0.008 – 0.010) - a critical property in production credit scoring.
- Specificity exceeds XGBoost (0.8571 vs. 0.7541 in Case 1), with better false positive control.

Limitations of HFA:

- Learns only linear mappings; cannot capture nonlinear relationships.
- Sensitive to the quality of the initial common subspace.
- Under large domain gap, P, Q mappings may push points into a shared cluster misaligned with the target distribution, inducing mild negative transfer.

6.3. DANN method evaluation

DANN yields divergent results between the two evaluation settings. In the hold-out ablation (Case 1 - target only), the target network encoder achieves $\text{AUC} = 0.7659\text{--}0.7681$ - surpassing XGBoost - demonstrating that the MLP architecture is well-suited to TDCCC data. However, Full DANN (Case 3) drops sharply to $\text{AUC} = 0.4655$ (Case 1) and 0.6167 (Case 2), while 5-fold CV yields $\text{AUC} = 0.5048 \pm 0.1103$ (Case 1) and 0.5111 ± 0.0944 (Case 2) - near the random classifier threshold with variance approximately 12 times larger than HFA.

Training logs confirm the failure mechanism: $\mathcal{L}_d \approx 0$ for most of the 4,000 steps - the domain classifier fails to learn any source/target discriminative signal, causing the GRL to backpropagate meaningless gradients to the encoder (GRL degeneracy). This is a direct manifestation of the theoretical warning of Zhao et al. (2019): when λ^* is large, forcing $\mathcal{H}$ -divergence to zero destroys the joint distribution. Case 2 reduces the severity of failure (+15.12 pp over Case 1) as feature selection narrows the domain gap, but not sufficiently to rescue the adversarial mechanism in the heterogeneous domain setting.

Advantages of DANN:

- Deep encoder captures complex nonlinear relationships (evidenced by Case 1 hold-out $\text{AUC} \sim 0.77$).
- GRL is elegant - only a single backward pass required.
- Warm-up and polynomial LR decay smoothly control adversarial signal strength.

Limitations of DANN:

- Extremely unstable when the two domain encoders have different input dimensions ($d_S \neq d_T$).
- $\mathcal{H}$ -divergence bound not directly applicable under label shift.
- Large fold-to-fold variance (± 0.10) makes results difficult to reproduce and unreliable in production.

6.4. HFA and DANN comparison

Table 25: HFA and DANN result comparison

Criterion	HFA	DANN
Model type	Linear (P, Q + LinearSVC)	Nonlinear (MLP + GRL)
AUC hold-out (Full method, Case 2)	0.7042	0.6167
KS hold-out (Full method, Case 2)	0.3595	0.2095
Δ vs. Case 1 (target only)	−0.34 pp	−15.14 pp

Criterion	HFA	DANN
AUC 5-fold CV (Case 2)	0.7218 ± 0.0098	0.5111 ± 0.0944
Convergence guarantee	Yes (Tseng, 2001, Th. 4.1)	No (non-convex)
Training cost	Low (CPU)	Moderate (CPU/GPU)
Interpretability	High (linear)	Low (deep)
Robustness under label shift	Low	Low (GRL collapse)
Fold-to-fold variance	Low (± 0.009)	High (± 0.094)

On the LC $\rightarrow$ TDCCC dataset pair, HFA outperforms DANN in stability and deployment practicality (hold-out AUC 0.7042 vs. 0.6167, variance approximately $10\times$ lower), even though neither surpasses the XGBoost baseline. Ablation results indicate that the majority of DANN's observed "benefit" comes from the MLP architecture (Case 1, target only: AUC ~ 0.77), not from the adversarial alignment mechanism - which is effectively disabled by GRL degeneracy. This suggests that for this heterogeneous domain pair, linear feature alignment (HFA) is the safer and more reliable choice compared to adversarial alignment.

6.5. Limitations

Four limitations warrant acknowledgment. First, due to computational resource constraints, HFA was tested with $d_c \in \{24, 64\}$ and DANN with `hidden_dim` = 128; larger common subspaces or deeper encoders may yield different results. Second, the hyperparameter space was not exhaustively explored (only one configuration per method); a grid search over η , λ , `hidden_dim`, and `n_outer` could improve AUC by 0.02–0.05 points according to recent benchmarks. Third, more advanced methods that explicitly address label shift - CDAN (Long et al., 2018, NeurIPS), MCD (Saito et al., 2018), Wasserstein DA - were not compared; these may overcome GRL degeneracy in the heterogeneous setting. Fourth, TDCCC is Taiwanese data from 2005; application to modern credit systems (e.g., Vietnam 2020+) requires additional validation for concept drift and population distribution shift.

6.6. Future research

Four viable research directions emerge from this work. First, apply HDA methods with explicit label shift handling - CDAN (Long et al., 2018, NeurIPS) with conditional alignment via multilinear conditioning, or IWDAN (Tachet et al., 2020, NeurIPS) combining importance weighting with DANN. Second, extend to multi-source HDA - combining Lending Club, Prosper Loans, Freddie Mac, and FICO data to obtain a source more representative of general credit risk. Third, validate on Vietnamese credit data (the target market of this thesis), where the domain gap with US/Taiwan sources may be even larger and feature availability lower. Fourth, integrate complementary techniques: pseudo-labeling (self-training on unlabeled target), source–target MixUp, or gradient surgery (GradVac, PCGrad) to reduce gradient conflict between the label loss and domain loss.

The experimental results affirm an important lesson established in the literature (Wang et al., 2019, CVPR; Rosenstein et al., 2005): transfer learning is not always beneficial, and explicitly testing for negative transfer through 3-case ablation must be a mandatory component of any transfer learning study - particularly in risk-critical domains such as credit scoring, where performance degradation can translate directly into real financial losses.

REFERENCES

- Achlioptas, D. (2003) 'Database-friendly random projections: Johnson-Lindenstrauss with binary coins', *Journal of Computer and System Sciences*, 66(4), pp. 671–687. doi: 10.1016/S0022-0000(03)00025-4.
- Altman, E. I. (1968) 'Financial ratios, discriminant analysis and the prediction of corporate bankruptcy', *Journal of Finance*, 23(4), pp. 589–609. doi: 10.1111/j.1540-6261.1968.tb00843.x.
- Altman, E. I. and Saunders, A. (1997) 'Credit risk measurement: Developments over the last 20 years', *Journal of Banking & Finance*, 21(11–12), pp. 1721–1742. doi: 10.1016/S0378-4266(97)00036-8.
- Ba, J. L., Kiros, J. R. and Hinton, G. E. (2016) 'Layer normalization', *arXiv preprint*, arXiv:1607.06450. doi: 10.48550/arXiv.1607.06450.
- Babaev, D., Savchenko, M., Tuzhilin, A. and Umerenkov, D. (2019) 'E.T.-RNN: Applying deep learning to credit loan applications', in *Proceedings of the 25th ACM SIGKDD Conference*. New York: ACM, pp. 2183–2190. doi: 10.1145/3292500.3330693.
- Baesens, B., Van Gestel, T., Viaene, S., Stepanova, M., Suykens, J. and Vanthienen, J. (2003) 'Benchmarking state-of-the-art classification algorithms for credit scoring', *Journal of the Operational Research Society*, 54(6), pp. 627–635. doi: 10.1057/palgrave.jors.2601545.
- Basel Committee on Banking Supervision (2017) *Basel III: Finalising Post-Crisis Reforms*. Basel: Bank for International Settlements.
- Beaver, W. H. (1966) 'Financial ratios as predictors of failure', *Journal of Accounting Research*, 4, pp. 71–111. doi: 10.2307/2490171.
- Ben-David, S., Blitzer, J., Crammer, K. and Pereira, F. (2007) 'Analysis of representations for domain adaptation', in *Advances in Neural Information Processing Systems 19*. Cambridge, MA: MIT Press.
- Ben-David, S., Blitzer, J., Crammer, K., Kulesza, A., Pereira, F. and Vaughan, J. W. (2010) 'A theory of learning from different domains', *Machine Learning*, 79(1–2), pp. 151–175. doi: 10.1007/s10994-009-5152-4.
- Bertsekas, D. P. (1999) *Nonlinear Programming*. 2nd edn. Belmont, MA: Athena Scientific. ISBN: 978-1-886529-00-7.
- Borisov, V., Leemann, T., Seßler, K., Haug, J., Pawelczyk, M. and Kasneci, G. (2024) 'Deep neural networks and tabular data: A survey', *IEEE Transactions on Neural Networks and Learning Systems*, 35(6), pp. 7499–7519. doi: 10.1109/TNNLS.2022.3229161.
- Bouthillier, X., Laurent, C. and Vincent, P. (2019) 'Unreproducible research is reproducible', in *Proceedings of the 36th International Conference on Machine Learning (ICML)*. PMLR.
- Boyd, S. and Vandenberghe, L. (2004) *Convex Optimization*. Cambridge: Cambridge University Press. ISBN: 978-0-521-83378-3.

- Bracke, P., Datta, A., Jung, C. and Sen, S. (2019) 'Machine learning explainability in finance: An application to default risk analysis', *Bank of England Staff Working Paper*, No. 816. doi: 10.2139/ssrn.3435104.
- Brier, G. W. (1950) 'Verification of forecasts expressed in terms of probability', *Monthly Weather Review*, 78(1), pp. 1–3. doi: 10.1175/1520-0493(1950)078<0001:VOFEWP>2.0.CO;2.
- Cao, K., Wei, C., Gaidon, A., Aréchiga, N. and Ma, T. (2019) 'Learning imbalanced datasets with label-distribution-aware margin loss', in *Advances in Neural Information Processing Systems* 32.
- Chapelle, O. (2007) 'Training a support vector machine in the primal', *Neural Computation*, 19(5), pp. 1155–1178. doi: 10.1162/neco.2007.19.5.1155.
- Chen, T. and Guestrin, C. (2016) 'XGBoost: A scalable tree boosting system', in *Proceedings of the 22nd ACM SIGKDD Conference*. New York: ACM, pp. 785–794. doi: 10.1145/2939672.2939785.
- Chen, W.-Y., Hsu, T.-M. H., Tsai, Y.-H. H., Wang, Y.-C. F. and Chen, M.-S. (2016) 'Transfer neural trees for heterogeneous domain adaptation', in *European Conference on Computer Vision (ECCV)*. Cham: Springer, pp. 399–414. doi: 10.1007/978-3-319-46454-1_25.
- Cortes, C. and Vapnik, V. (1995) 'Support-vector networks', *Machine Learning*, 20(3), pp. 273–297. doi: 10.1007/BF00994018.
- Cox, D. R. (1958) 'The regression analysis of binary sequences', *Journal of the Royal Statistical Society: Series B*, 20(2), pp. 215–242. doi: 10.1111/j.2517-6161.1958.tb00292.x.
- Dai, W., Yang, Q., Xue, G.-R. and Yu, Y. (2007) 'Boosting for transfer learning', in *Proceedings of the 24th International Conference on Machine Learning (ICML)*. New York: ACM, pp. 193–200. doi: 10.1145/1273496.1273521.
- Duan, L., Xu, D. and Tsang, I. W. (2012) 'Learning with augmented features for heterogeneous domain adaptation', in *Proceedings of the 29th International Conference on Machine Learning (ICML)*.
- Durand, D. (1941) *Risk Elements in Consumer Instalment Financing*. New York: National Bureau of Economic Research.
- Eisenbeis, R. A. (1977) 'Pitfalls in the application of discriminant analysis in business, finance, and economics', *Journal of Finance*, 32(3), pp. 875–900. doi: 10.1111/j.1540-6261.1977.tb01995.x.
- Engelmann, J. and Lessmann, S. (2021) 'Conditional Wasserstein GAN-based oversampling of tabular data for imbalanced learning', *Expert Systems with Applications*, 174, 114582. doi: 10.1016/j.eswa.2021.114582.
- Fan, R.-E., Chang, K.-W., Hsieh, C.-J., Wang, X.-R. and Lin, C.-J. (2008) 'LIBLINEAR: A library for large linear classification', *Journal of Machine Learning Research*, 9, pp. 1871–1874.

- Fawcett, T. (2006) 'An introduction to ROC analysis', *Pattern Recognition Letters*, 27(8), pp. 861–874. doi: 10.1016/j.patrec.2005.10.010.
- Ganin, Y. and Lempitsky, V. (2015) 'Unsupervised domain adaptation by backpropagation', in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. PMLR.
- Ganin, Y., Ustinova, E., Ajakan, H., Germain, P., Larochelle, H., Laviolette, F., Marchand, M. and Lempitsky, V. (2016) 'Domain-adversarial training of neural networks', *Journal of Machine Learning Research*, 17(59), pp. 1–35.
- Géron, A. (2019) *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*. 2nd edn. Sebastopol, CA: O'Reilly.
- Glorot, X. and Bengio, Y. (2010) 'Understanding the difficulty of training deep feedforward neural networks', in *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*. PMLR.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. and Bengio, Y. (2014) 'Generative adversarial networks', in *Advances in Neural Information Processing Systems* 27.
- Gretton, A., Borgwardt, K. M., Rasch, M. J., Schölkopf, B. and Smola, A. (2012) 'A kernel two-sample test', *Journal of Machine Learning Research*, 13, pp. 723–773.
- Halevy, A., Norvig, P. and Pereira, F. (2009) 'The unreasonable effectiveness of data', *IEEE Intelligent Systems*, 24(2), pp. 8–12. doi: 10.1109/MIS.2009.36.
- Hand, D. J. (2009) 'Measuring classifier performance: A coherent alternative to the area under the ROC curve', *Machine Learning*, 77(1), pp. 103–123. doi: 10.1007/s10994-009-5119-5.
- Hand, D. J. and Henley, W. E. (1997) 'Statistical classification methods in consumer credit scoring: A review', *Journal of the Royal Statistical Society: Series A*, 160(3), pp. 523–541. doi: 10.1111/1467-985X.00106.
- Hastie, T., Tibshirani, R. and Friedman, J. (2009) *The Elements of Statistical Learning*. 2nd edn. New York: Springer. doi: 10.1007/978-0-387-84858-7.
- He, K., Zhang, X., Ren, S. and Sun, J. (2015) 'Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification', in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. pp. 1026–1034. doi: 10.1109/ICCV.2015.123.
- Hsieh, C.-J., Chang, K.-W., Lin, C.-J., Keerthi, S. S. and Sundararajan, S. (2008) 'A dual coordinate descent method for large-scale linear SVM', in *Proceedings of the 25th International Conference on Machine Learning (ICML)*. New York: ACM, pp. 408–415. doi: 10.1145/1390156.1390208.
- (BŌ SUNG) Hsu, C.-W., Chang, C.-C. and Lin, C.-J. (2003) *A Practical Guide to Support Vector Classification*. Technical Report. Taipei: National Taiwan University.

- Johnson, W. B. and Lindenstrauss, J. (1984) 'Extensions of Lipschitz mappings into a Hilbert space', in *Contemporary Mathematics*, 26, pp. 189–206. doi: 10.1090/conm/026/737400.
- Kaufman, S., Rosset, S. and Perlich, C. (2012) 'Leakage in data mining: Formulation, detection, and avoidance', *ACM SIGKDD Explorations Newsletter*, 14(1), pp. 15–22. doi: 10.1145/2382577.2382579.
- Khandani, A. E., Kim, A. J. and Lo, A. W. (2010) 'Consumer credit-risk models via machine-learning algorithms', *Journal of Banking & Finance*, 34(11), pp. 2767–2787. doi: 10.1016/j.jbankfin.2010.06.001.
- King, G. and Zeng, L. (2001) 'Logistic regression in rare events data', *Political Analysis*, 9(2), pp. 137–163. doi: 10.1093/oxfordjournals.pan.a004868.
- Kingma, D. P. and Ba, J. (2015) 'Adam: A method for stochastic optimization', in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. arXiv:1412.6980.
- Kohavi, R. (1995) 'A study of cross-validation and bootstrap for accuracy estimation and model selection', in *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*.
- Krzanowski, W. J. and Hand, D. J. (2009) *ROC Curves for Continuous Data*. Boca Raton, FL: Chapman and Hall/CRC.
- Lessmann, S., Baesens, B., Seow, H.-V. and Thomas, L. C. (2015) 'Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research', *European Journal of Operational Research*, 247(1), pp. 124–136. doi: 10.1016/j.ejor.2015.05.030.
- Liu, X., Liu, Z., Wang, G., Cai, Z. and Zhang, H., 2017. Ensemble transfer learning algorithm. *Ieee Access*, 6, pp.2389-2396.
- Long, M., Cao, Y., Wang, J. and Jordan, M. I. (2015) 'Learning transferable features with deep adaptation networks', in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. PMLR.
- Long, M., Cao, Z., Wang, J. and Jordan, M. I. (2018) 'Conditional adversarial domain adaptation', in *Advances in Neural Information Processing Systems 31*.
- Mansour, Y., Mohri, M. and Rostamizadeh, A. (2009) 'Domain adaptation: Learning bounds and algorithms', in *Proceedings of the 22nd Conference on Learning Theory (COLT)*. doi: 10.1007/978-3-642-04414-4_4.
- Nguyen, T. et al. (2024) 'Development of a credit scoring model using machine learning for Vietnamese banks', *Advances and Applications in Statistics*, 92(1). doi: 10.17654/0972361725006.
- Pan, S. J. and Yang, Q. (2010) 'A survey on transfer learning', *IEEE Transactions on Knowledge and Data Engineering*, 22(10), pp. 1345–1359. doi: 10.1109/TKDE.2009.191.

- Pascanu, R., Mikolov, T. and Bengio, Y. (2013) 'On the difficulty of training recurrent neural networks', in *Proceedings of the 30th International Conference on Machine Learning (ICML)*. PMLR.
- Pedregosa, F. et al. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- Platt, J. C. (1999) 'Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods', in Smola, A. J. et al. (eds) *Advances in Large Margin Classifiers*. Cambridge, MA: MIT Press, pp. 61–74.
- Rosenstein, M. T., Marx, Z., Kaelbling, L. P. and Dietterich, T. G. (2005) 'To transfer or not to transfer', in *NIPS 2005 Workshop on Transfer Learning*.
- Saito, K., Watanabe, K., Ushiku, Y. and Harada, T. (2018) 'Maximum classifier discrepancy for unsupervised domain adaptation', in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 3723–3732. doi: 10.1109/CVPR.2018.00392.
- Schölkopf, B. and Smola, A. J. (2002) *Learning with Kernels*. Cambridge, MA: MIT Press. ISBN: 978-0-262-19475-4.
- Siddiqi, N. (2012) *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. Hoboken, NJ: Wiley. doi: 10.1002/9781119201731.
- State Bank of Vietnam (2021) *CIC công bố mô hình chấm điểm tín dụng thẻ nhân 2.0*. Hanoi: SBV.
- State Bank of Vietnam (2024) *Draft Decree on Sandbox Mechanism in the Banking Sector*. Hanoi: SBV.
- Thomas, L. C. (2009) *Consumer Credit Models: Pricing, Profit and Portfolios*. Oxford: Oxford University Press.
- Tobback, E. and Martens, D. (2019) 'Retail credit scoring using fine-grained payment data', *Journal of the Royal Statistical Society: Series A*, 182(4), pp. 1227–1246. doi: 10.1111/rssa.12469.
- Tsai, C.-F. and Wu, J.-W. (2008) 'Using neural network ensembles for bankruptcy prediction and credit scoring', *Expert Systems with Applications*, 34(4), pp. 2639–2649. doi: 10.1016/j.eswa.2007.05.019.
- Tseng, P. (2001) 'Convergence of a block coordinate descent method for nondifferentiable minimization', *Journal of Optimization Theory and Applications*, 109(3), pp. 475–494. doi: 10.1023/A:1017501703105.
- Tzeng, E., Hoffman, J., Saenko, K. and Darrell, T. (2017) 'Adversarial discriminative domain adaptation', in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 7167–7176. doi: 10.1109/CVPR.2017.316.

- Tzeng, E., Hoffman, J., Zhang, N., Saenko, K. and Darrell, T. (2014) 'Deep domain confusion: Maximizing for domain invariance', *arXiv preprint*, arXiv:1412.3474. doi: 10.48550/arXiv.1412.3474.
- Wang, C. and Mahadevan, S. (2011) 'Heterogeneous domain adaptation using manifold alignment', in *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI)*.
- Wang, Z., Dai, Z., Póczos, B. and Carbonell, J. (2019) 'Characterizing and avoiding negative transfer', in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. pp. 11293–11302. doi: 10.1109/CVPR.2019.01155.
- Weiss, K., Khoshgoftaar, T. M. and Wang, D. (2016) 'A survey of transfer learning', *Journal of Big Data*, 3(1), 9. doi: 10.1186/s40537-016-0043-6.
- Xu, F. and Zhang, R., 2025. Explainable domain adaptation learning framework for credit scoring in internet finance through adversarial transfer learning and ensemble fusion model. *Mathematics*, 13(7), p.1045. doi: 10.3390/math13071045.
- Yang, Q., Zhang, Y., Dai, W. and Pan, S. J. (2020) *Transfer Learning*. Cambridge: Cambridge University Press.
- Yeh, I.-C. and Lien, C.-h. (2009) 'The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients', *Expert Systems with Applications*, 36(2), pp. 2473–2480. doi: 10.1016/j.eswa.2007.12.020.
- Yosinski, J., Clune, J., Bengio, Y. and Lipson, H. (2014) 'How transferable are features in deep neural networks?', in *Advances in Neural Information Processing Systems* 27.
- Youden, W. J. (1950) 'Index for rating diagnostic tests', *Cancer*, 3(1), pp. 32–35. doi: 10.1002/1097-0142(1950)3:1<32::AID-CNCR2820030106>3.0.CO;2-3.
- Zhao, H., Combes, R. T. d., Zhang, K. and Gordon, G. (2019) 'On learning invariant representations for domain adaptation', in *Proceedings of the 36th International Conference on Machine Learning (ICML)*. PMLR.

Appendix A - Mathematical Derivations

A.1. From SVM Dual to HFA

This section traces the derivation from the classical linear SVM (Cortes & Vapnik, 1995) to the HFA objective (Yang, 2020, Eq. 6.8), making every assumption, every Lagrangian step, and every design choice explicit.

Step 1 - Primal SVM. Given data (x_i, y_i) with $x_i \in \mathbb{R}^d$, $y_i \in \{-1, +1\}$, find the hyperplane (w, b) that separates the two classes with maximum margin. Formalized with squared-hinge loss:

$$\min_{w, b} \frac{1}{2} \|w\|^2 + C \sum_i \max(0, 1 - y_i(w^\top x_i + b))^2$$

Step 2 - Kernelization and Lagrangian dual (Schölkopf & Smola, 2002, §7):

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

with $\alpha_i \in [0, C]$ and $\sum_i \alpha_i y_i = 0$. In HFA, the kernel K is replaced by a kernel learned over the shared representation space, leading to Eq. 6.8.

Step 3 - Common subspace for heterogeneous domains. Duan, Xu, and Tsang (2012) posit the existence of two linear projections $P \in \mathbb{R}^{d_c \times d_s}$ and $Q \in \mathbb{R}^{d_c \times d_r}$. Features are augmented as:

$$\varphi_s(x_s) = [Px_s; x_s; 0_{d_t}], \varphi_t(x_t) = [Qx_t; 0_{d_s}; x_t]$$

This augmentation ensures the leading block is of equal dimension and directly comparable; the original raw features are preserved in their own block to prevent information loss; and the zero-padding ensures domain separation (Yang, 2020, Chapter 6, §6.3.2, pp. 142–144).

Step 4 - HFA objective. Substituting φ into the dual SVM and simplifying (Yang, 2020, §6.3.3, Eq. 6.5–6.8) yields the final HFA objective (Eq. 6.8).

A.2. Frobenius Projection Proof

Boyd and Vandenberghe (2004, §8.1.1) establish that the set $\{M: \|M\|_F \leq r\}$ is a closed convex set in $\mathbb{R}^{m \times n}$. For any M outside the set, the Euclidean projection onto it has the closed-form solution:

$$\text{proj}_c(M) = \frac{r}{\|M\|_F} \cdot M \text{ when } \|M\|_F > r,$$

and returns M otherwise.

The choice of Frobenius norm (over nuclear norm or spectral norm) rests on three points: Frobenius has a smooth gradient and a closed-form projection; it corresponds to L2 regularization on the vectorized matrix; and it aligns with the theory of Ridge regression and Tikhonov regularization (Hastie, 2009, §3.4).

A.3. BCD Convergence Proof

The HFA algorithm applies the Block Coordinate Descent (BCD) strategy: fix (P, Q) , solve for $\mathbf{w}$ via LinearSVC; fix $\mathbf{w}$, update (P, Q) via projected subgradient. Tseng (2001) guarantees that if (i) the objective is continuous in each block variable and convex in each block when the others are fixed, (ii) the feasible set is a Cartesian product of closed convex sets, and (iii) each sub-step is non-increasing in the objective, then every accumulation point of the sequence is a stationary point of f .

In HFA: condition (i) holds because with (P, Q) fixed, the objective over $\mathbf{w}$ is a standard squared-hinge SVM (convex and smooth in $\mathbf{w}$); with $\mathbf{w}$ fixed, the objective over (P, Q) is a linear combination of convex squared-hinge terms plus a convex Frobenius regularizer. Condition (ii) holds because the Frobenius ball is a closed convex set and $\mathbf{w}$ is unconstrained. Condition (iii) holds because LinearSVC solves to epsilon tolerance, and the projected subgradient with small step size monotonically decreases the objective (Bertsekas, 1999).

A.4. Squared-Hinge and Subgradient

The classical hinge $\max(0, 1 - z)$ is non-differentiable at $z = 1$, producing a subgradient of either $\{0\}$ or $\{-1\}$; the squared-hinge $\max(0, 1 - z)^2$ is differentiable everywhere and has a gradient linear in $(1 - z)$, enabling smoother BCD convergence (Chapelle, 2007):

$$\ell(z) = \max(0, 1 - z)^2, \ell'(z) = -2\max(0, 1 - z)$$

Applying the chain rule to $\ell(y_i \mathbf{w}^\top \varphi_s(\mathbf{x}_i))$ and differentiating with respect to P yields the formula implemented in `_grad_PQ()`:

$$\frac{\partial \ell_i}{\partial P} = -2C \max(0, 1 - y_i \mathbf{w}^\top \varphi_s(\mathbf{x}_i)) \cdot y_i \cdot \mathbf{w}_c \mathbf{x}_{s,i}^\top$$

where $\mathbf{w}_c$ is the portion of $\mathbf{w}$ corresponding to the common-subspace block.

The full three-step chain rule derivation is as follows. Let $z_i = \mathbf{w}^\top \varphi_s(\mathbf{x}_i) + b$ be the margin score and $m_i = y_i z_i$ the signed margin. The loss for sample i is:

$$\ell_i = \max(0, 1 - m_i)^2$$

$$\text{Step 1: } \frac{\partial \ell_i}{\partial m_i} = -2\max(0, 1 - m_i)$$

$$\text{Step 2: } \frac{\partial m_i}{\partial z_i} = y_i \text{ [since } m_i = y_i z_i \text{]}$$

$$\text{Step 3: } \frac{\partial z_i}{\partial P} = \mathbf{w}_c \mathbf{x}_{s,i}^\top \text{ [outer product } \in \mathbb{R}^{d_c \times d_s} \text{]}$$

Combining and adding the regularizer:

$$\frac{\partial L}{\partial P} = \sum_{i \in \mathcal{A}_S} \frac{\partial \ell_i}{\partial m_i} \cdot \frac{\partial m_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial P} + \lambda_P P$$

The factor of 2 in $-2\max(0, 1 - m_i)$ can be absorbed into the learning rate η . The implementation keeps the factor of 2 explicit and adjusts $\eta = 8 \times 10^{-4}$ instead of 10^{-3} to compensate. The active set $\mathcal{A}_S = \{i: 1 - y_i z_i > 0\}$ ensures the gradient is zero for correctly classified samples outside the margin - the core sparsity property of SVM that makes the algorithm efficient (Hsieh et al., 2008).

A.5. Random Gaussian Projection and the Johnson–Lindenstrauss Lemma

The initialisation procedure sets P and Q using the Gaussian distribution $\mathbf{R} \sim \mathcal{N}(0, 1/d_c)$. This choice derives from the Johnson–Lindenstrauss lemma (Johnson & Lindenstrauss, 1984, *Contemporary Mathematics*) and the finite-sample version of Achlioptas (2003, Theorem 1.1):

$$\Pr[(1 - \varepsilon) \|\mathbf{x}\|^2 \leq \|\mathbf{R}\mathbf{x}\|^2 \leq (1 + \varepsilon) \|\mathbf{x}\|^2] \geq 1 - 2\exp(-d_c \varepsilon^2 / 8)$$

for all $\mathbf{x} \in \mathbb{R}^d$, with $\mathbf{R}$ having i.i.d. entries distributed as $\mathcal{N}(0, 1/d_c)$.

Concretely: with $d_c = 64$ and $\varepsilon = 0.5$, the norm-preservation probability per point is approximately $1 - 2\exp(-2) \approx 0.73$; with $d_c = 128$ it rises to approximately 0.97. The experimental setup selects $d_c \in \{64, 128\}$ to preserve initial geometry before BCD fine-tunes. Furthermore, random initialization avoids symmetry breaking - initializing with zeros produces zero gradients at the first step (Glorot & Bengio, 2010).

A.6. Platt Calibration Derivation

Platt (1999, *Advances in Large Margin Classifiers*, §5) transforms the SVM decision score $f(\mathbf{x}) \in \mathbb{R}$ into a probability via sigmoid:

$$P(y = 1 | \mathbf{x}) = \sigma(Af(\mathbf{x}) + B) = \frac{1}{1 + \exp[-(Af(\mathbf{x}) + B)]}$$

Parameters (A, B) are found by MLE on the calibration set. The log-likelihood is:

$$L(A, B) = \sum_i [t_i \log p_i + (1 - t_i) \log (1 - p_i)]$$

with $p_i = \sigma(Af_i + B)$, $t_i = (y_i + 1)/2$. In practice, logistic regression with balanced class weights is used for calibration, equivalent to Platt's MLE combined with King–Zeng (2001) re-weighting for class imbalance.

Optimizing $L(A, B)$ via gradient descent, the partial derivatives (with $p_i = \sigma(Af_i + B)$) are:

$$\frac{\partial L}{\partial A} = \sum_i (p_i - t_i) f_i, \quad \frac{\partial L}{\partial B} = \sum_i (p_i - t_i)$$

These are the standard logistic regression gradients - precisely the problem that sklearn's LogisticRegression solves. Setting $C = 10^6$ corresponds to negligible regularization, approximating pure MLE. When class imbalance is present, `class_weight='balanced'` multiplies each $(p_i - t_i)$ by its corresponding weight before summation - equivalent to re-weighting the log-likelihood (King & Zeng, 2001).

Note that A is typically negative ($-A > 0$) since the SVM score $f(\mathbf{x}) = \mathbf{w}^\top \varphi(\mathbf{x}) + b$ is positive for the positive class and negative for the negative class. For $\sigma(Af + B) \rightarrow 1$ as f increases, we need $A < 0$ (Platt, 1999, notes A typically lies between -1 and -3 for standard SVM hinge loss; for squared-hinge, A is typically close to -1).

A.7. KS Statistic and Youden's J

The Kolmogorov–Smirnov statistic in credit scoring is the maximum distance between the CDFs of predicted default probabilities for the good and bad groups (Thomas, 2009, §4.5):

$$KS = \sup_t |F_{\text{good}}(t) - F_{\text{bad}}(t)|$$

Youden's J statistic (Youden, 1950, *Cancer* 3:32–35) is:

$$J(t) = \text{Sensitivity}(t) + \text{Specificity}(t) - 1 = \text{TPR}(t) - \text{FPR}(t)$$

Theorem: $KS = \max_t J(t)$. Equivalently, the Youden-optimal threshold coincides with the KS-optimal threshold (Krzanowski & Hand, 2009). Therefore, a single thresholding procedure is applied for both KS computation and evaluation metrics.

The choice of $t^* = \arg \max_t (\text{TPR}(t) - \text{FPR}(t))$ over $\arg \max F_1$ is motivated by the asymmetric cost structure of credit scoring: false negatives (extending credit to borrowers who will default) are approximately 5–10 times more costly than false positives (Hand, 2009). The KS threshold symmetrically balances TPR and FPR and is the de facto standard under Basel III (BCBS, 2017, §4.5).

A.8. Adam Optimizer Analysis

Kingma and Ba (2015) define Adam as:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = m_t / (1 - \beta_1^t), \hat{v}_t = v_t / (1 - \beta_2^t)$$

$$\theta_t = \theta_{t-1} - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

Default parameters $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$. The DANN implementation adopts these default values, following the recommendation of Kingma and Ba (2015, §6).

Adam is preferred for DANN for three reasons: (i) adversarial loss gradients reverse direction abruptly due to the GRL (sign flip with increasing λ) - Adam is more stable than SGD under such perturbations; (ii) per-parameter adaptive learning rates accommodate the heterogeneous encoders whose gradient scales differ between source and target; and (iii) bias correction prevents under-stepping during the warm-up phase.

A.9. H-divergence Bound Proof Sketch

Ben-David, Blitzer, Crammer, Kulesza, Pereira, and Vaughan (2010) prove:

$$\varepsilon_T(h) \leq \varepsilon_S(h) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(\mathcal{D}_S, \mathcal{D}_T) + \lambda^*$$

where $\varepsilon_T(h)$ and $\varepsilon_S(h)$ are the risks of hypothesis h on target and source domains; $d_{\mathcal{H}\Delta\mathcal{H}}$ is the $\mathcal{H}$ -divergence (measured by the best discriminator in the symmetric difference hypothesis class); and $\lambda^* = \min_{h \in \mathcal{H}} [\varepsilon_S(h) + \varepsilon_T(h)]$ is the combined error of the ideal joint hypothesis. The proof is based on the triangle inequality for L1 norm on distributions and the Pinsker inequality.

Reducing $d_{\mathcal{H}\Delta\mathcal{H}}$ is equivalent to tightening the bound on ε_T . The direct mechanism is to learn a representation $g(\mathbf{x})$ such that the distributions $g(\mathcal{D}_S)$ and $g(\mathcal{D}_T)$ cannot be distinguished by the domain classifier - the core adversarial idea of Ganin (2015, §3): if the domain classifier performs no better than random, then $d_{\mathcal{H}\Delta\mathcal{H}}(g(\mathcal{D}_S), g(\mathcal{D}_T)) \approx 0$, tightening the bound.

The GRL operationalizes this through a min-max game: the encoder seeks to maximize the domain classifier's loss (via gradient reversal), while the domain classifier continues to minimize its own loss. In practice, the signal of good convergence is the domain classifier accuracy converging to approximately 0.5 (Ganin, 2016, Figure 3).

Appendix B - Experimental Design Rationale

B.1. LinearSVC

The LinearSVC implementation offers two solvers: liblinear (Fan, Chang, Hsieh, Wang & Lin, 2008) and L-BFGS. Liblinear is selected for three reasons: it optimizes L2 + squared-hinge with $O(1/\varepsilon)$ convergence guaranteeing ε -optimality (Hsieh, 2008); it natively supports `class_weight='balanced'`, equivalent to re-weighting positive and negative samples, and it scales well with large `n_features` - TDCCC after augmentation has $d_c + d_s + d_T \approx 64 + 23 + 120(\text{or } 37) = 207(\text{or } 124)$ features.

The principal hyperparameter is C (regularization, default 1.0). The experiment evaluates $C \in \{0.1, 1.0, 10\}$ and selects via CV

B.2. 5-fold cross-validation

Kohavi (1995) demonstrates that 10-fold CV has lower bias than 5-fold but higher variance. For TDCCC with $n = 30000$, the bias difference is negligible (below 0.5% AUC). 5-fold halves wall-time - particularly relevant for HFA and DANN, which are computationally expensive: LinearSVC over 6000 samples $\times$ 2 domains $\times$ 5 outer iterations amounts to 60000 SVM fits, approximately 30 minutes wall-clock.

StratifiedKfold (Pedregosa, 2011) ensures the class ratio in each fold approximates the population distribution - critical for imbalanced data such as TDCCC (22% default).

B.3. Z-score

Z-score normalization (StandardScaler in sklearn): $x' = (x - \mu)/\sigma$. The choice rests on three points: linear-kernel SVM is sensitive to feature scale (Hsu, 2003, §2.2); Adam's moment estimate $\hat{v}_t$ is proportional to σ^2 of the gradient - when feature scales differ significantly, the effective learning rate varies between weights, causing instability; Min-Max normalization is only warranted when data has a fixed bounded range (e.g., image pixels $\in [0, 255]$), whereas tabular financial features have unbounded distributions.

Reproducibility note: StandardScaler is fitted exclusively on the training fold, then the fitted scaler is applied to the test fold, following the best practice of Géron (2019, Chapter 2 Pipelines) to prevent data leakage.

B.4. Leakage Removal

Lending Club contains approximately 145 columns, many of which are post-loan variables populated after the loan is disbursed: `total_pymnt`, `recoveries`, `last_pymnt_d`, and similar fields. Using any of these would allow the model to learn target leakage (Kaufman, 2012). All post-loan columns are removed based on Lending Club's official data dictionary. This is the standard practice established by Bracke (2019, Bank of England Staff Working Paper 816, §4.2) - the anchor benchmark for ML-based credit scoring.

B.5. Random seed and stability

A fixed random seed (`SEED = 42`) is applied consistently across all random number generators (NumPy, PyTorch, and scikit-learn) to ensure full reproducibility. The choice of 42 is purely conventional and carries no mathematical significance - what matters is consistency. According to Bouthillier (2019), variance across seeds can reach 0.5–1% AUC; experimental studies should report the seed to allow reproduction.

CUDA determinism note: setting `torch.backends.cudnn.deterministic = True` reduces throughput by approximately 10%. This trade-off is accepted to guarantee absolute reproducibility, which is prioritised over speed for a research deliverable.

Appendix C - Notation

Symbol	Meaning
$\mathbf{x}_s, \mathbf{x}_t$	Source and target feature vectors
d_s, d_t	Source and target feature dimensionalities
d_c	Common subspace dimensionality
n_s, n_t	Source and target sample sizes
$P \in \mathbb{R}^{d_c \times d_s}$	Source projection into the shared space
$Q \in \mathbb{R}^{d_c \times d_t}$	Target projection into the shared space
φ_s, φ_t	Source and target augmentation functions
w, b	SVM hyperplane in the augmented space
w_c	First d_c components of w (common block)
ξ_i	Slack variable (squared-hinge)
C	SVM regularization coefficient
λ_P, λ_Q	Soft Frobenius regularization coefficients
r_P, r_Q	Frobenius ball radii (hard constraint)
η	Learning rate for projected gradient
n_{outer}	Number of BCD outer iterations
$\mathcal{A}$	Active set (margin violators)
A, B	Platt scaling parameters
t^*	KS-optimal classification threshold
$\lambda(p)$	Adversarial weight schedule in DANN
$\eta(p)$	Learning rate schedule in DANN
γ_t	Target classifier weight = $\sqrt{n_s/n_t}$
$\varepsilon_s, \varepsilon_T$	Source and target risk

Symbol	Meaning
$d_{\mathcal{H}\Delta\mathcal{H}}$	$\mathcal{H}$ -divergence
TPR, FPR	True and False Positive Rates
KS	Kolmogorov–Smirnov statistic
AUC	Area Under the ROC Curve